Laboratory 1A - Introduction to Digital Signal Processing

Table of Contents

QUESTION 1 - Sinusoidal Signal Generation (2 pts) 1
QUESTION 2 - Cosinusoidal Signal Generation (2 pts) 3
QUESTION 3 - Basics of Sampling (3 pts) 5
QUESTION 4 - Basics of Sampling (3 pts) 9
QUESTION 5 - Manipulating Biomedical Signals (2 pts) 13
Phonocardiogram signal 13
Electroencephalogram signal 15
QUESTION 6 - Manipulating Biomedical Signals (2 pts) 17
Electrocardiogram signal 17
Electromyogram signal 19
QUESTION 7 - Elementary Discrete-Time Signals and properties (6 pts) 21
Elementary Signals and Transformations 21

QUESTION 1 - Sinusoidal Signal Generation (2 pts)

A sinusoidal signal is defined as

$$x(t) = A \cdot \sin(2\pi f\ t + \phi\ ) + r(t)$$

where:

- A : amplitude
- f : frequency
- t : time
- ϕ : phase shift
- $r(t)$: noise

Goal: Simulate in MATLAB a continuous signal with a duration of 0.5 seconds, composed of three sinusoidals with different frequencies, amplitudes and phase shifts with added white noise.

For this part, you are required to do the following:

a) **(1 pt)** Generate three sinusoidal signals with different frequencies using the `sin()` function. Assign each a different amplitude and phase.

Define the time vector for the simulation with 1000 samples (use `linspace`).

Use the following parameters:

- First signal - $A = 2, f = 5\,\text{Hz},\ \phi = 0.5\pi$
- Second signal - $A = 1, f = 15\,\text{Hz},\ \phi = 0.25\pi$

- Third signal - $A = 0.5, f = 30\text{Hz}, \phi = 0$

```
t= linspace(0,0.5,1000);

x1 = 2*sin(2*pi*5*t + 0.5*pi);
x2 = 1*sin(2*pi*15*t + 0.25*pi);
x3 = 0.5*sin(2*pi*30*t + 0);
```

b) **(1 pt)** Sum the three sinusoidal signals and add white noise. Generate the noise using `randn(N,1)` and add it to the signal. Consider $\sigma = 0,1$.

$$r(t) = \sigma.\text{randn}(N, 1)$$

Then graph the resulting signal using `plot()`. Add a title and label both axes using `xlabel and ylabel`.

Note: N is the total sample size and can be defined as the signal duration multiplied by the sampling frequency.

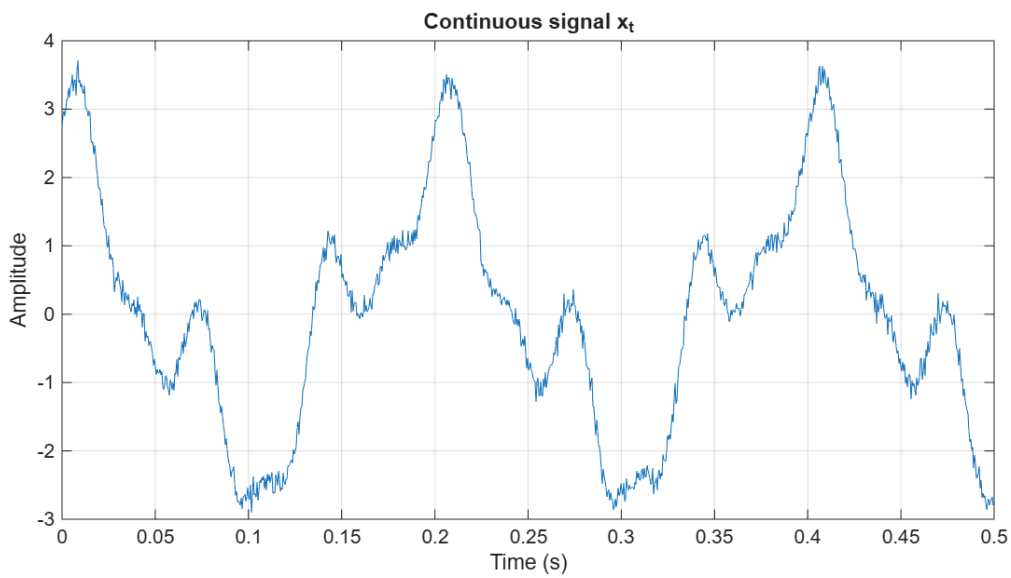
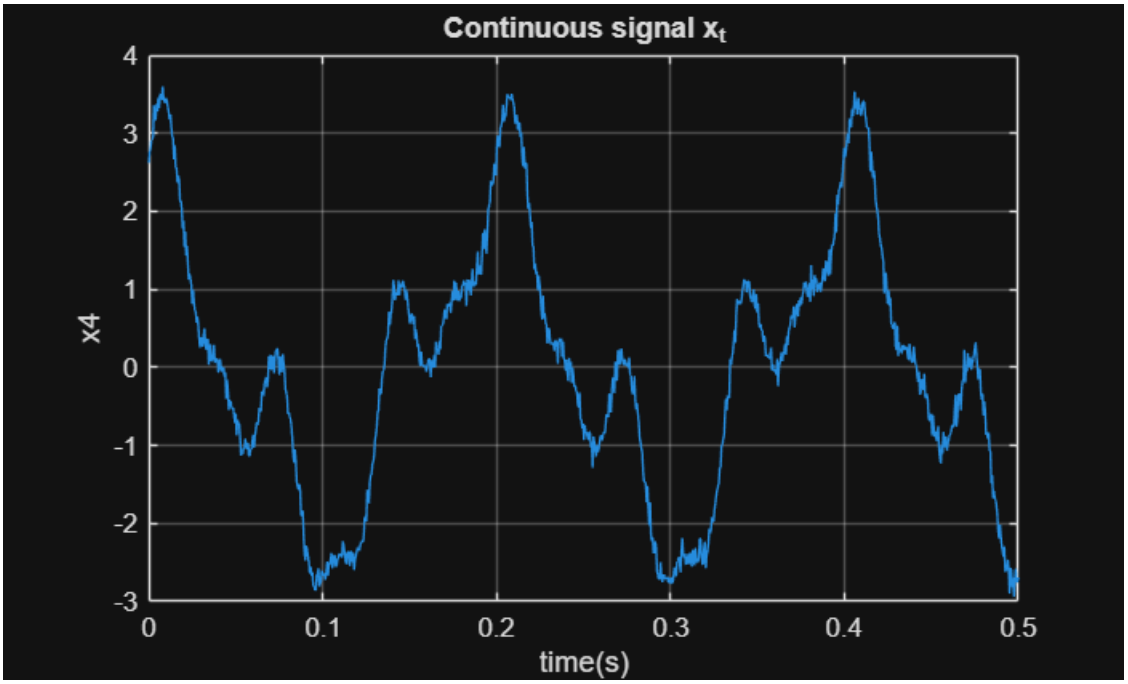


Figure 1. Expected results.

```
r= 0.1*randn(1000,1);
r=r.';

x4= x1+x2+x3+r;
plot(t, x4); xlabel("time(s)"); ylabel("x4"); title("Continuous signal x_{t}");
grid on;
```



QUESTION 2 - Cosinusoidal Signal Generation (2 pts)

```
clear
```

A cosinusoidal signal is defined as

$$x(t) = A \cdot \cos(2\pi f\ t + \phi\ ) + r(t)$$

where:

- A : amplitude
- f : frequency
- t : time
- ϕ : phase shift
- $r(t)$: noise

Goal: Simulate in MATLAB a continuous signal with a duration of 0.25 seconds, composed of four cosinusoidals with different frequencies, amplitudes and phase shifts with added white noise.

For this part, you are required to do the following:

a) **(1 pt)** Generate four cosinusoidal signals with different frequencies using the `cos()` function. Assign each a different amplitude and phase.

Define the time vector for the simulation with 1000 samples (use `linspace`)

Use the following parameters:

- First signal - $A = 3, f = 12\text{Hz}, \phi = 0.75\pi$
- Second signal - $A = 4, f = 4\text{Hz}, \phi = \frac{3}{7}\pi$
- Third signal - $A = 2, f = 50\text{Hz}, \phi = 1.25\pi$
- Fourth signal - $A = 1, f = 100\text{Hz}, \phi = 0$

```
t= linspace(0,0.25,1000);
y1 = 3*cos(2*pi*12*t + 0.75*pi);
y2 = 4*cos(2*pi*4*t + (3/7)*pi);
y3 = 2*cos(2*pi*50*t + 1.25*pi);
y4 = 1*cos(2*pi*100*t + 0);
```

b) **(1 pt)** Sum the four cosinusoidal signals and add white noise. Generate the noise using `randn(N,1)` and add it to the signal. Consider $\sigma = 0,2$.

$$r(t) = \sigma.\text{randn}(N, 1)$$

Then graph the resulting signal using `plot()`. Add a title and label both axes using `xlabel and ylabel`.

Note: N is the total sample size and can be defined as the signal duration multiplied by the sampling frequency.

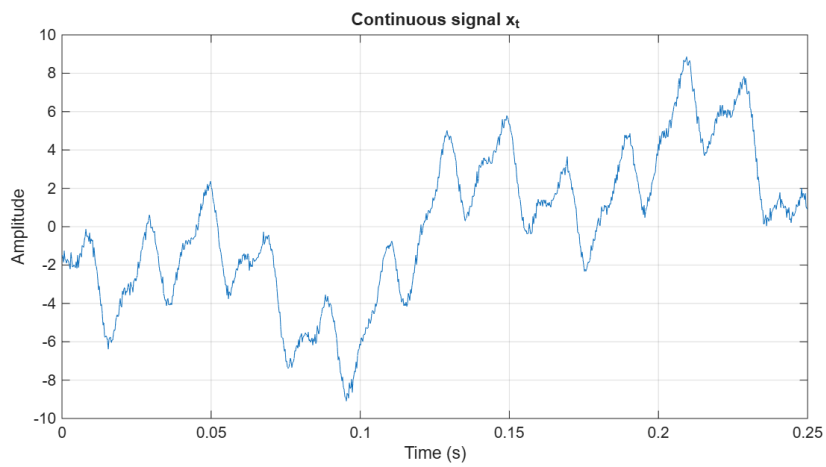
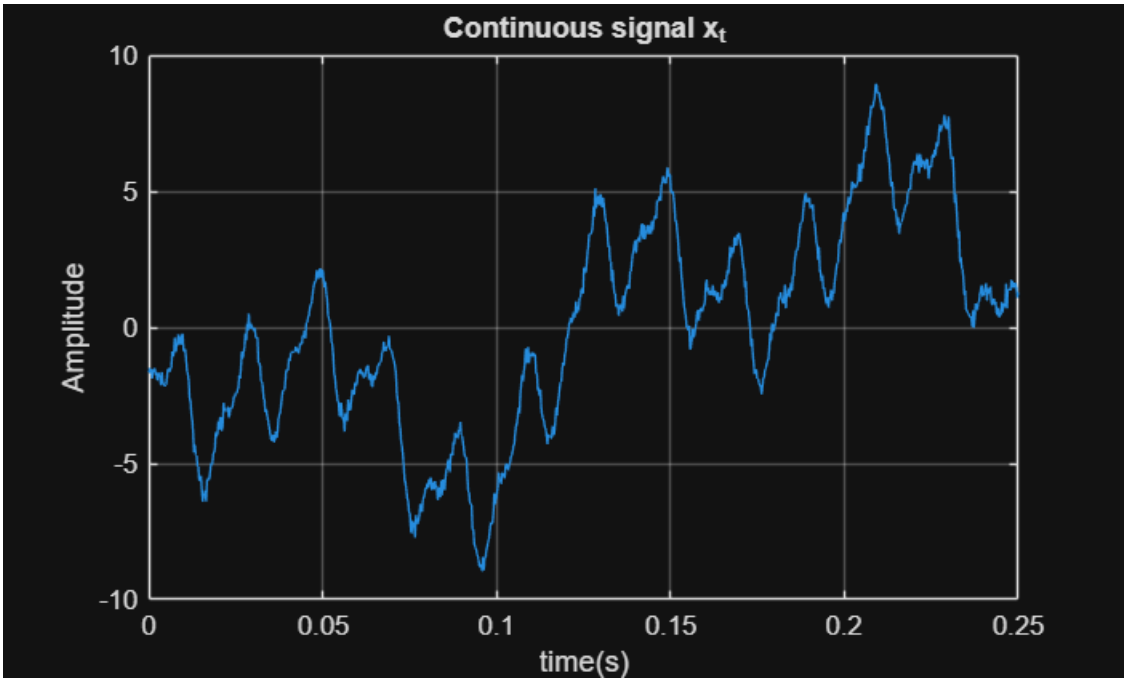


Figure 2. Expected results.

```
r1= 0.2*randn(1000,1);
r1=r1.';
y5 = y1 + y2 + y3 + y4 + r1;
plot(t,y5); xlabel("time(s)"); ylabel("Amplitude"); title("Continuous signal
x_{t}");grid on;
```



QUESTION 3 - Basics of Sampling (3 pts)

```
clear
```

An analog signal from a physical system must be digitized to be processed by a computer. This process depends on the sampling frequency, which determines how faithfully the original signal will be represented.

Goal: Analyze the effect of different sampling frequencies on a composite sinusoidal signal.

For this part, you are required to do the following:

a) **(1 pt)** Define a simulated continuous signal of 0.25 seconds duration, composed of the sum of two sinusoidal of frequencies 10 Hz and 60 Hz, respectively, both with unit amplitude and no phase. Use a time vector generated with `linspace` (use 1000 samples). Plot the simulated continuous signal using `plot()`.

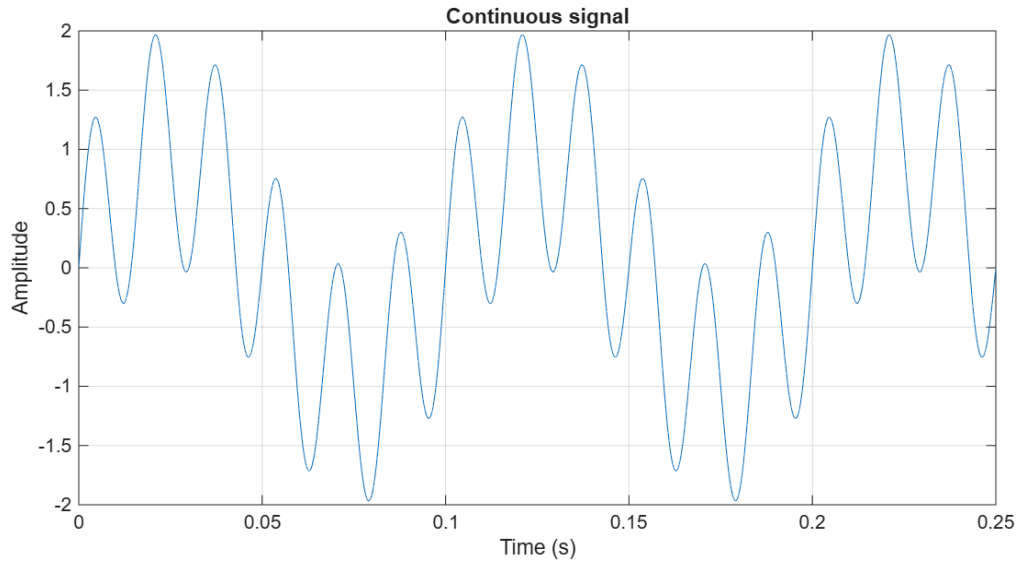
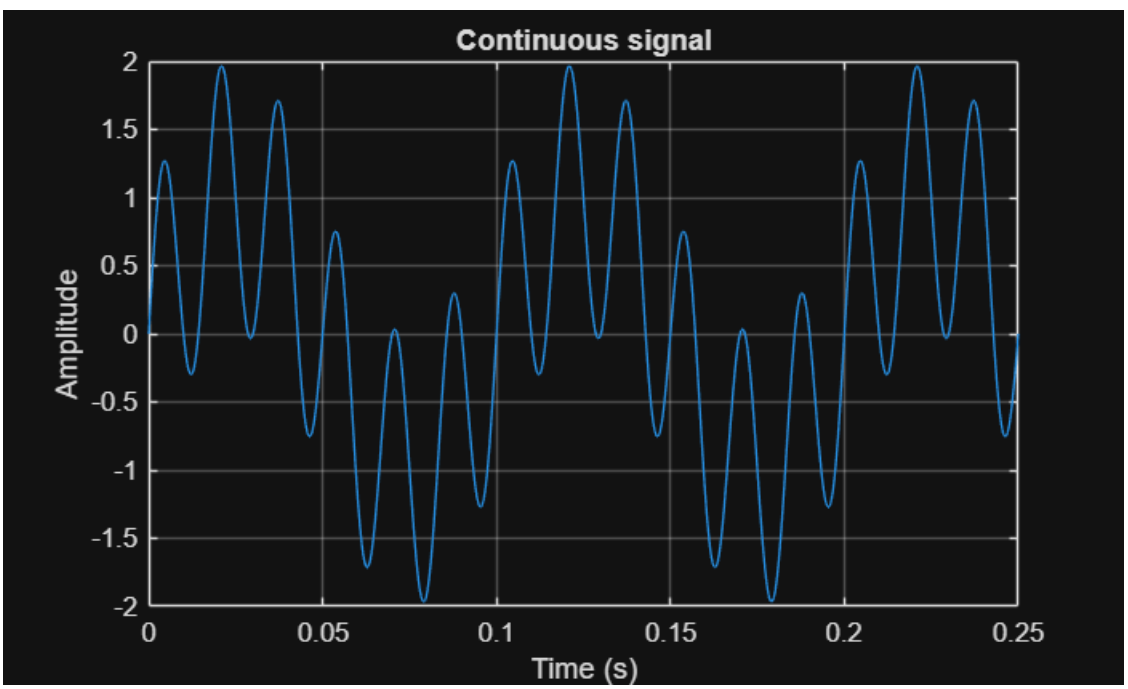


Figure 3. Expected results.

```
t2= linspace(0,0.25,1000);
z1= 1*sin(2*pi*10*t2+0);
z2= 1*sin(2*pi*60*t2+0);
z3= z1+z2;
plot(t2,z3);xlabel("Time (s)");ylabel("Amplitude"); title("Continuous signal");grid
on;
```



Now we will compute and plot the sampled signal from the previously defined continuous signal, consider the sampling frequency of $F_s = 500\,\mathrm{Hz}$ and the total time duration $t_{\mathrm{dur}} = 0.25s$.

b) **(1 pt)** Given the sampling frequency and the total duration, compute the sampling period ($T_s = \frac{1}{F_s}$), the total number of samples ($N_m = \frac{t_{\text{dur}}}{T_s}$), and define the sample index vector ($n = 0 : N_m - 1$).

Compute the discrete version of the signal using the sample index vector n. Then plot the discrete signal and the continuous signal on the same figure using stem(). Use the time vector as the x-axis.

Note: Use the following relation:

$$t = n\,T_s$$

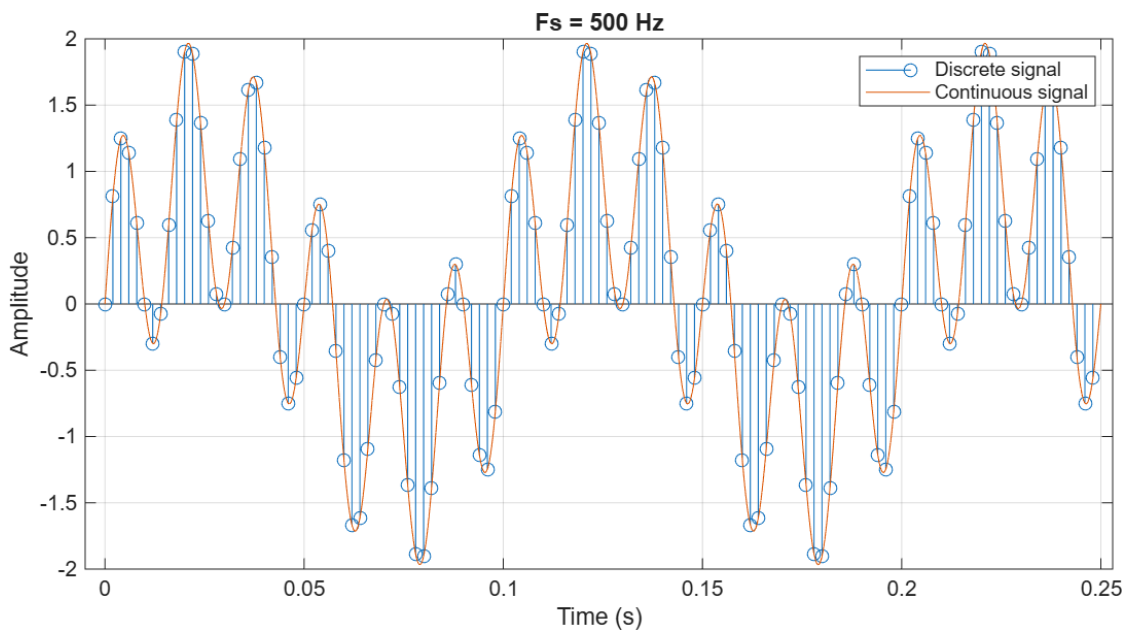
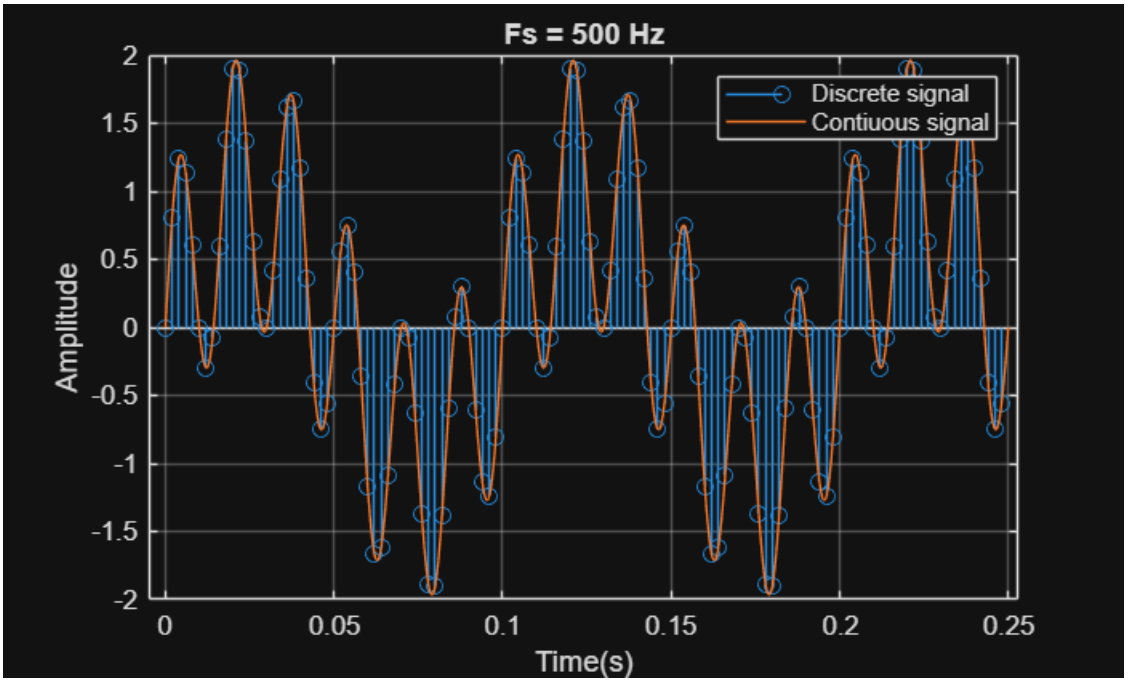


Figure 4. Expected results.

```
T= 1/500;
N= 0.25/T;
n1=0:N-1;
t3=n1*T;
m1= 1*sin(2*pi*10*t3+0);
m2= 1*sin(2*pi*60*t3+0);
m3=m1+m2;
figure;
stem(t3,m3);
hold on;
plot(t2,z3);xlabel("Time(s)");ylabel("Amplitude"); title("Fs = 500 Hz");grid
on;legend("Discrete signal","Contiuous signal");
hold off;
```



c) **(1 pt)** Repeat the step (b) using a sampling frequency of $F_s = 80\,\mathrm{Hz}$. Which sampling frequency is best at preserving the original signal shape?

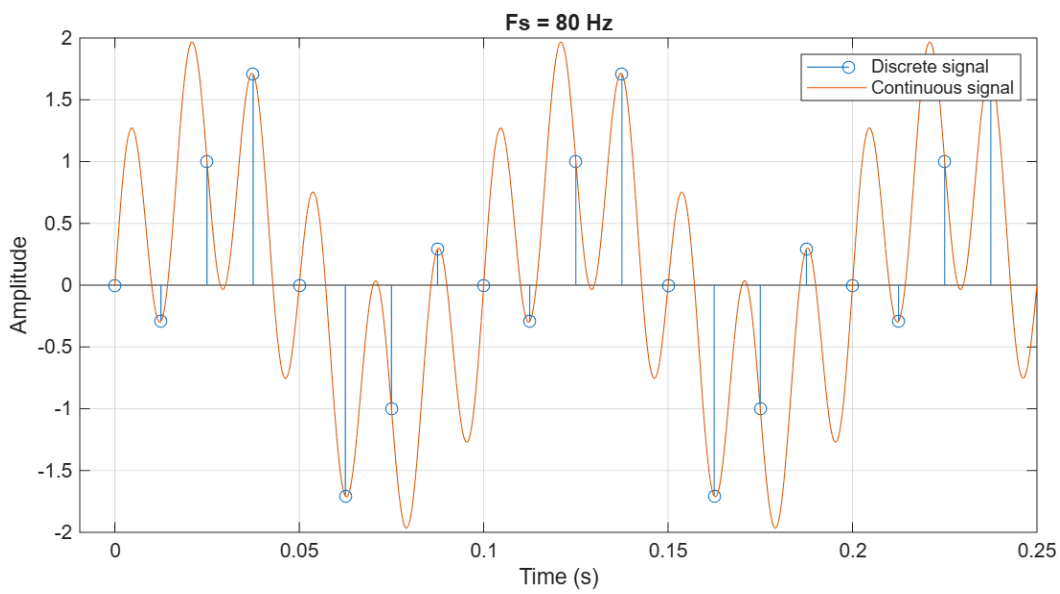
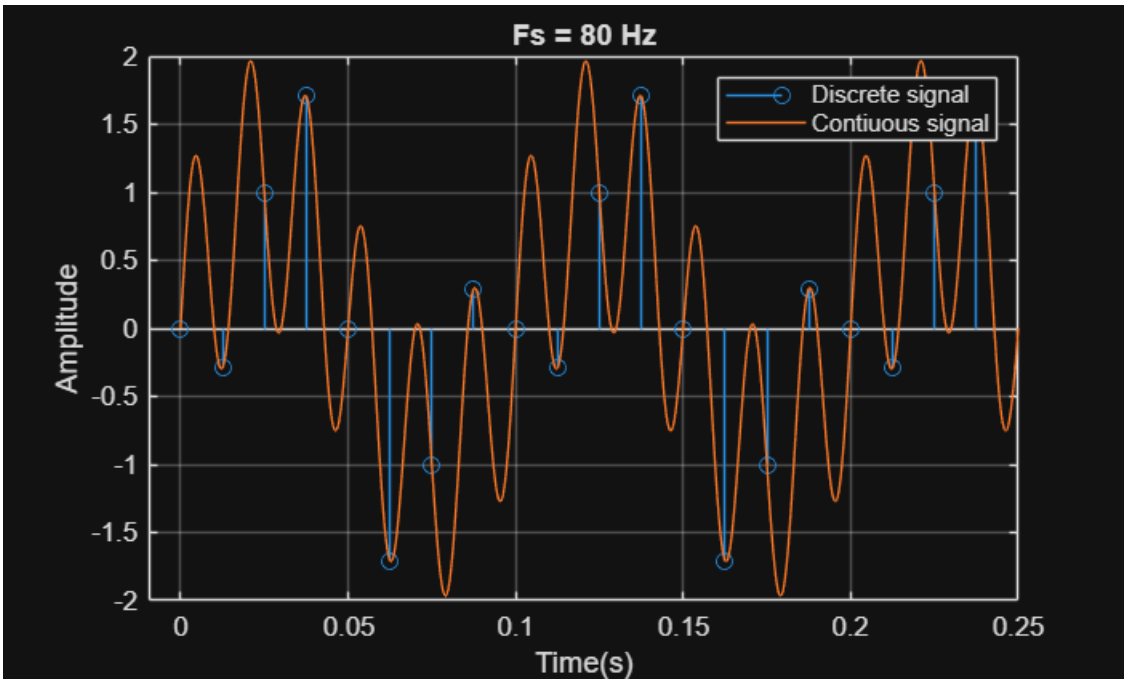


Figure 5. Expected results.

```
T1= 1/80;
N1= 0.25/T1;
n2=0:N1-1;
t3=n2*T1;
h1= 1*sin(2*pi*10*t3+0);
h2= 1*sin(2*pi*60*t3+0);
h3=h1+h2;
figure;
stem(t3,h3);
```

```
hold on;
plot(t2,z3);xlabel("Time(s)");ylabel("Amplitude"); title("Fs = 80 Hz");grid
on;legend("Discrete signal","Contiuous signal");
hold off;
```




```
%Picture with frecuency equal 500Hz is best at preserving the original
%signal shape because the higher the frequency, the clearer the original
%signal becomes.
```

QUESTION 4 - Basics of Sampling (3 pts)

```
clear
```

An analog signal from a physical system must be digitized to be processed by a computer. This process depends on the sampling frequency, which determines how faithfully the original signal will be represented.

Goal: Analyze the effect of different sampling frequencies on a composite sinusoidal signal.

For this part, you are required to do the following:

a) **(1 pt)** Define a simulated continuous signal of 0.5 seconds duration, composed of the sum of two sinusoidal of frequencies 45 Hz and 80 Hz, respectively, both with unit amplitude and no phase. Use a time vector generated with `linspace` (use 1000 samples). Plot the simulated continuous signal using `plot()`.

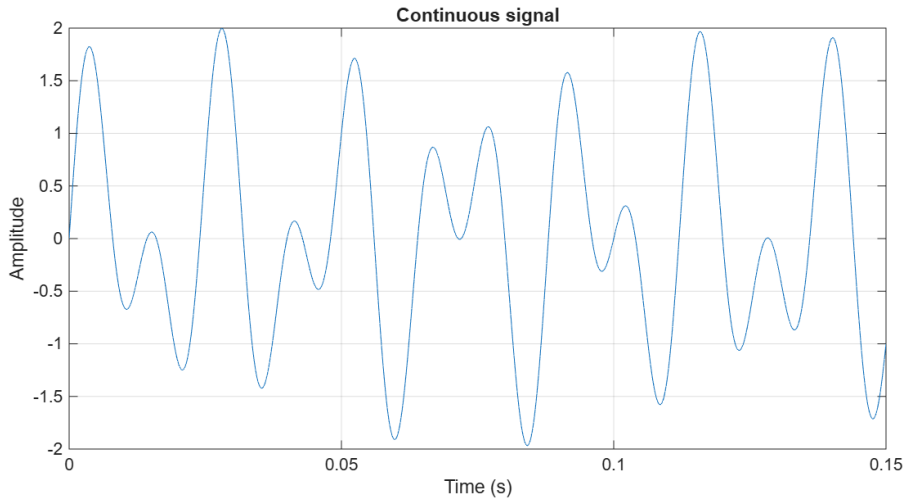
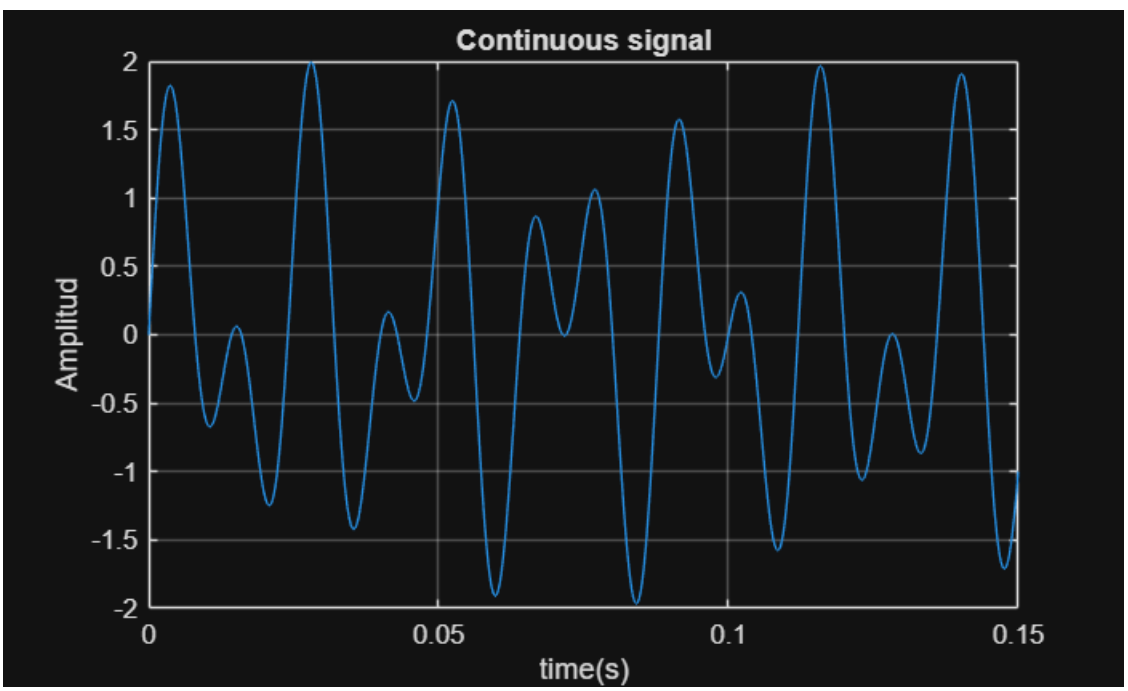


Figure 6. Expected results.

```
%Mistake 0.5 -> 0.15
t= linspace(0,0.15,1000);
x1=1*sin(2*pi*45*t+0);
x2=1*sin(2*pi*80*t+0);
x3= x1+x2;
plot(t,x3); xlabel("time(s)"); ylabel("Amplitud"); title("Continuous signal");grid
on;
```



Now we will compute and plot the sampled signal from the previously defined continuous signal, consider the sampling frequency of $F_s = 800\,\mathrm{Hz}$ and the total time duration $t_{\mathrm{dur}} = 0.15s$.

b) **(1 pt)** Given the sampling frequency and the total duration, compute the sampling period ($T_s = \frac{1}{F_s}$), the total number of samples ($N_m = \frac{t_{\text{dur}}}{T_s}$), and define the sample index vector ($n = 0 : N_m - 1$).

Compute the discrete version of the signal using the sample index vector n. Then plot the signal using stem(). Use the time vector as the x-axis.

Note: Use the following relation:

$$t = n\,T_s$$

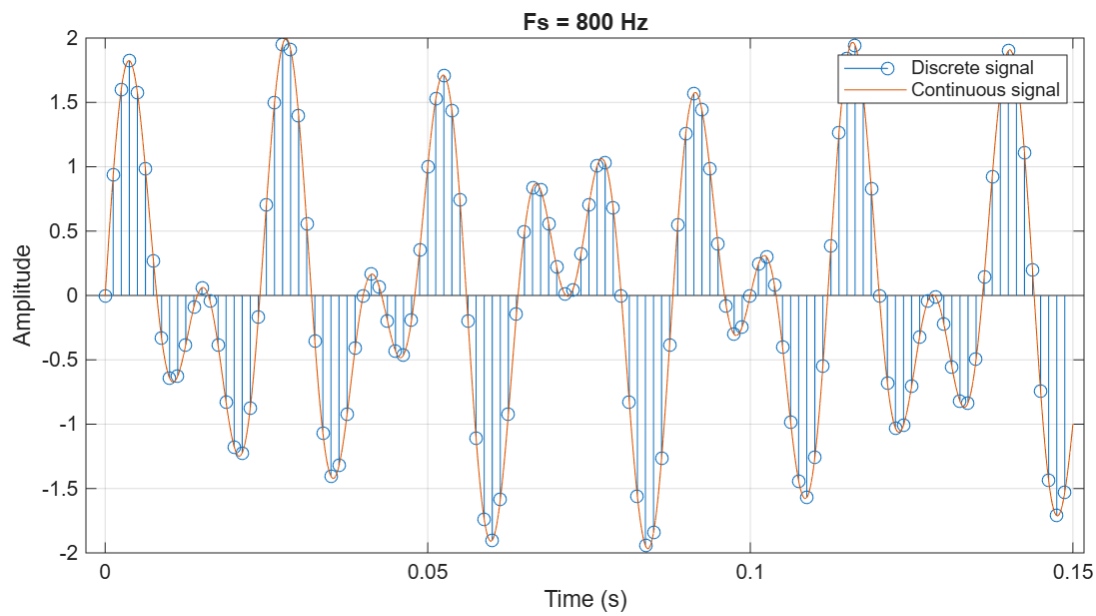
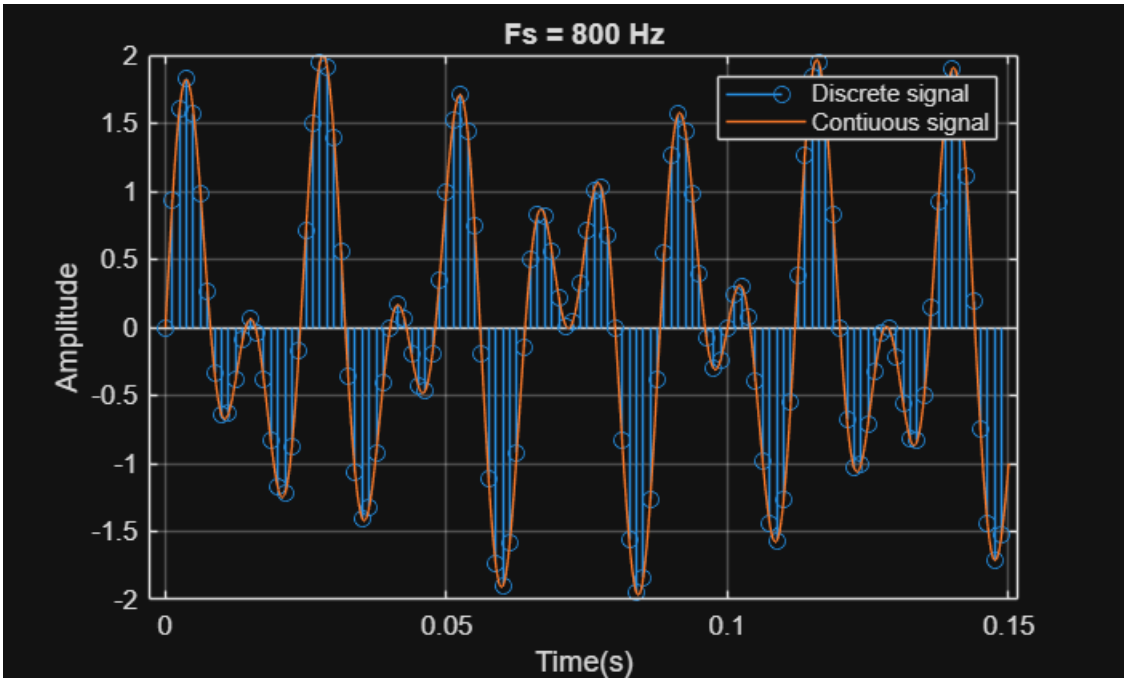


Figure 7. Expected results.

```
Ts=1/800;
td=0.15;
N=td/Ts;
n1=0:N-1;
t1=n1*Ts;
y1=1*sin(2*pi*45*t1+0);
y2=1*sin(2*pi*80*t1+0);
y3= y1+y2;
stem(t1,y3);
hold on;
plot(t,x3);xlabel("Time(s)");ylabel("Amplitude"); title("Fs = 800 Hz");grid
on;legend("Discrete signal","Contiuous signal");
hold off;
```



c) **(1 pt)** Repeat the step (b) using a sampling frequency of $F_s = 150\text{Hz}$. Which sampling frequency is best at preserving the original signal shape?

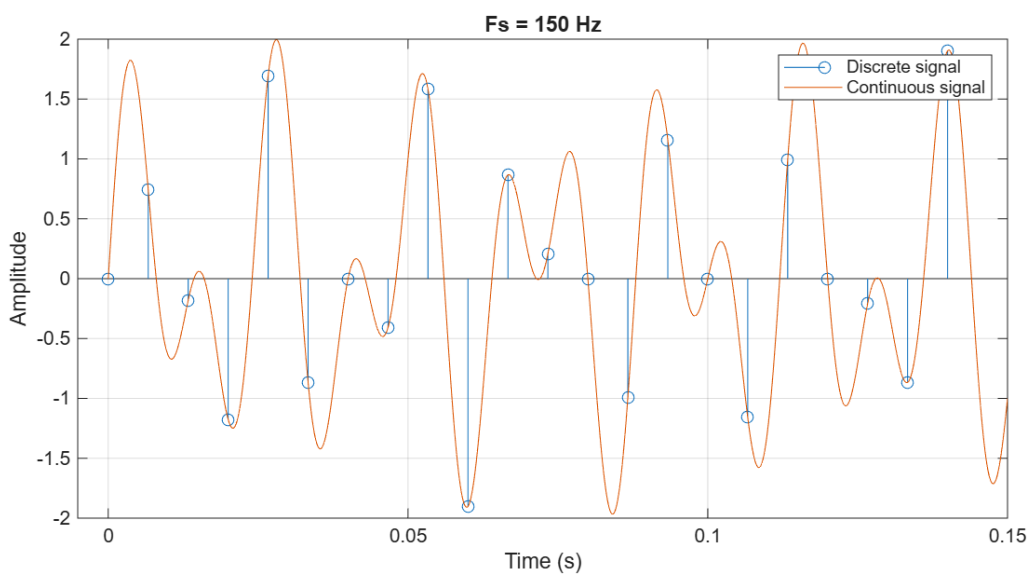
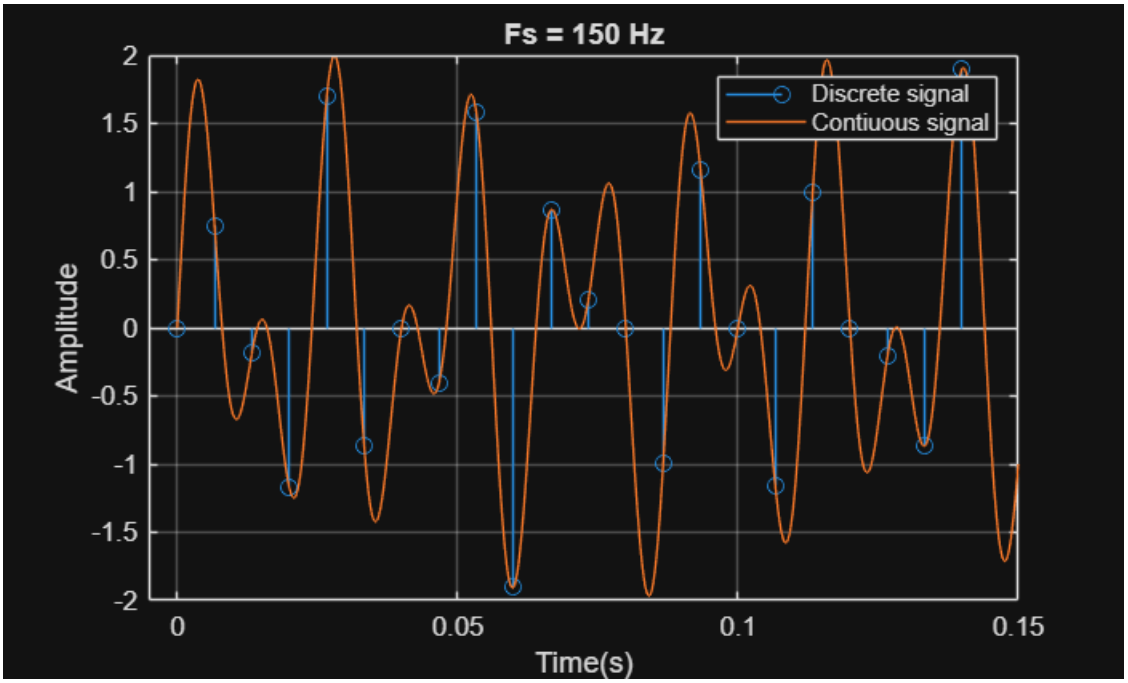


Figure 8. Expected results.

```
Ts=1/150;
td=0.15;
N=td/Ts;
n1=0:N-1;
t1=n1*Ts;
```

```
y1=1*sin(2*pi*45*t1+0);
y2=1*sin(2*pi*80*t1+0);
y3= y1+y2;
stem(t1,y3);
hold on;
plot(t,x3);xlabel("Time(s)");ylabel("Amplitude"); title("Fs = 150 Hz");grid
on;legend("Discrete signal","Contiuous signal");
hold off;
```




```
%Picture with frecuency equal 800Hz is best at preserving the original
%signal shape because the higher the frequency, the clearer the original
%signal becomes.
```

QUESTION 5 - Manipulating Biomedical Signals (2 pts)

```
clear
```

Phonocardiogram signal

Phonocardiogram signals allow non-invasive monitoring of heart sounds, which can be indicative of various cardiovascular conditions. Analyzing these signals helps in detecting abnormalities in the heart's function.
A recording corresponding to a database of heart fetal sounds is stored in the file `PCG_fetal.wav`. This phonocardiogram (PCG) signal was recorded by a digital stethoscope. Analyze the provided PCG signal in the time-domain and obtain its graphical representation.

a) **(0.5 pts)** Use the `audioread()` function and store the PCG signal and its corresponding sampling frequency in two variables (see MATLAB documentation).

```
[y, Fs] = audioread('PCG_fetal.wav')
```

Obtain the following values:

- Sampling frequency used by the digital stethoscope (in **`kHz (kiloHertz)`** units).
- Sampling period of the recording (in **`us (microseconds)`** units).
- Time duration of the recording (in **`s (seconds)`** units).

NOTE: Since the PCG signal has been recorded in an audio format, it contains two channels of data (left and right). Consider a single channel (any of them).

```
[y, Fs] = audioread('PCG_fetal.wav');
```

b) **(0.5 pts)** Plot the PCG signal by using the `plot()` function, employ a small window (from 15 to 16.2 seconds). Customize it by setting `xlabel(), ylabel(), xlim(), ylim(), title()` properly. Can you recognize some pattern in the plotted signal? Find the heart-beat in beats-per-minute (BPM) of the fetus.

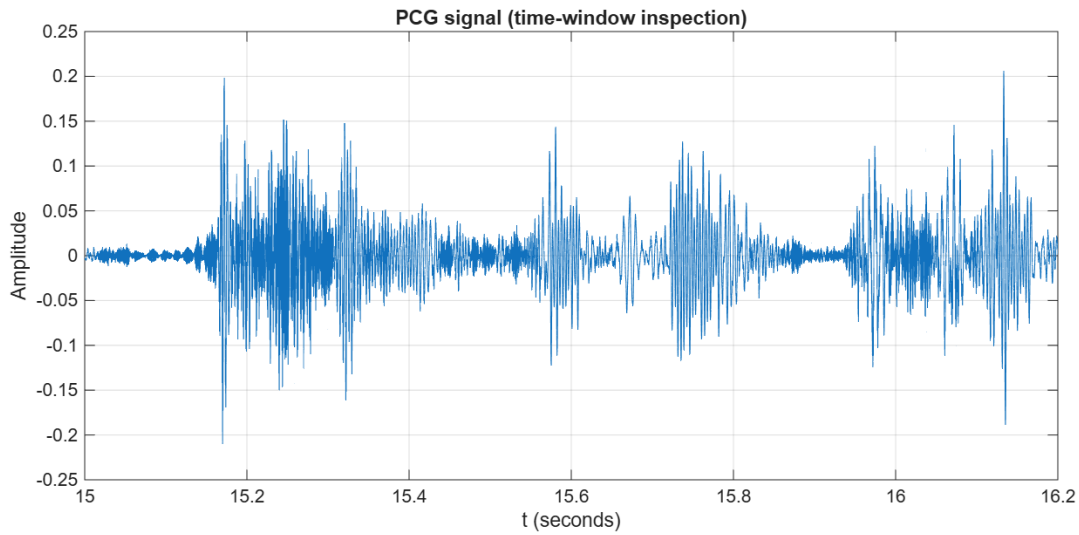
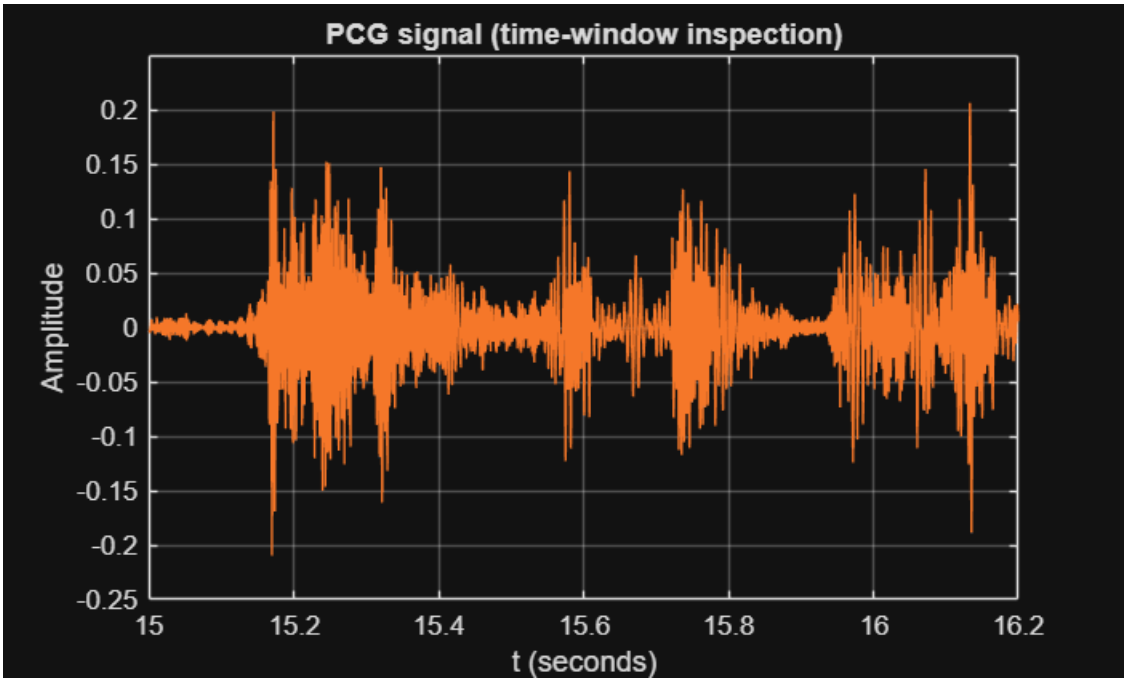


Figure 9. Expected results.

```
%We use the formula to find the time (t= Nm*Ts)
%Also remember this to find the period (Ts=1/Fs)
%And define the sample index vector (n=0:Nm-1)
Ts= 1/Fs;
Nm = length(y); % Number of samples in the audio signal
n1 = 0:Nm-1; % Sample index vector
t = n1 * Ts; % Time vector for the audio signal

figure, plot(t,y),xlabel("t (seconds)"),ylabel("Amplitude"),title("PCG signal (time-window inspection)"),xlim([15 16.2]),grid on;
```




```
%In the graph we can see 3 patterns.It repeats a lot of time so we conluded
%that it's the heart-beat.
%BPM = heart-beat/time * 60
% t = tf-fi= 16.2-15 = 1.2
% BPM = 3/1.2 *60=150
```

Electroencephalogram signal

```
clear
```

Electroencephalogram (EEG) signals allow for the non-invasive recording of the brain's electrical activity. Time-domain analysis is essential for identifying patterns associated with different brain states, such as rest, attention, or sleep, as well as potential neurological abnormalities.

Three EEG signals are available, stored in the file `EEG_signal.mat`, corresponding to an actual recording acquired using surface electrodes. The sampling frequency used is also available.

a) **(0.5 pts)** Load the `EEG_signal.mat` file using the `load()` function (see MATLAB documentation).

Obtain the following values:

- Sampling frequency (in **`kHz (kiloHertz)`** units).
- Sampling period of the recording (in **`ms (milliseconds)`** units).
- Time duration of the recording (in **`s (seconds)`** units).

NOTE: The file contains three channels of data. Consider a single channel (any of them).

```
s = load("EEG_signal.mat")
```

```
s = struct with fields:
```

```
EEG_signal: [3×11520 double]
        fs: 256
```

```
y=s.EEG_signal(2,:);
Fs= 256;
Fs_kHz1= 256/1000
```

```
Fs_kHz1 =
0.2560
```

```
%Ts= 1/Fs
Ts= 1/Fs;
Ts_ms1= Ts*1000
```

```
Ts_ms1 =
3.9062
```

```
%t= n*Ts
%n=0:Nm-1
Nm= length(y);
n1=0:Nm-1;
t= n1*Ts
```

```
t = 1×11520
         0    0.0039    0.0078    0.0117    0.0156    0.0195    0.0234    0.0273 ···
```

b) **(0.5 pts)** Plot the EEG signal by using the `plot()` function, employ a small window (from 0 to 20 seconds). Customize it by setting `xlabel()`, `ylabel()`, `xlim()`, `ylim()`, `title()` properly.

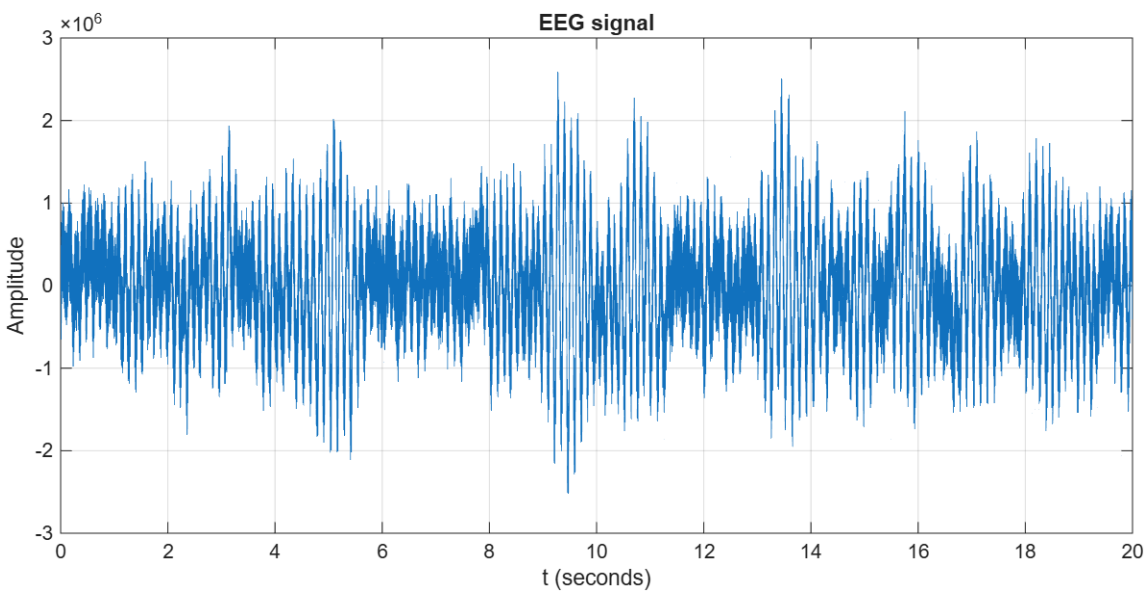
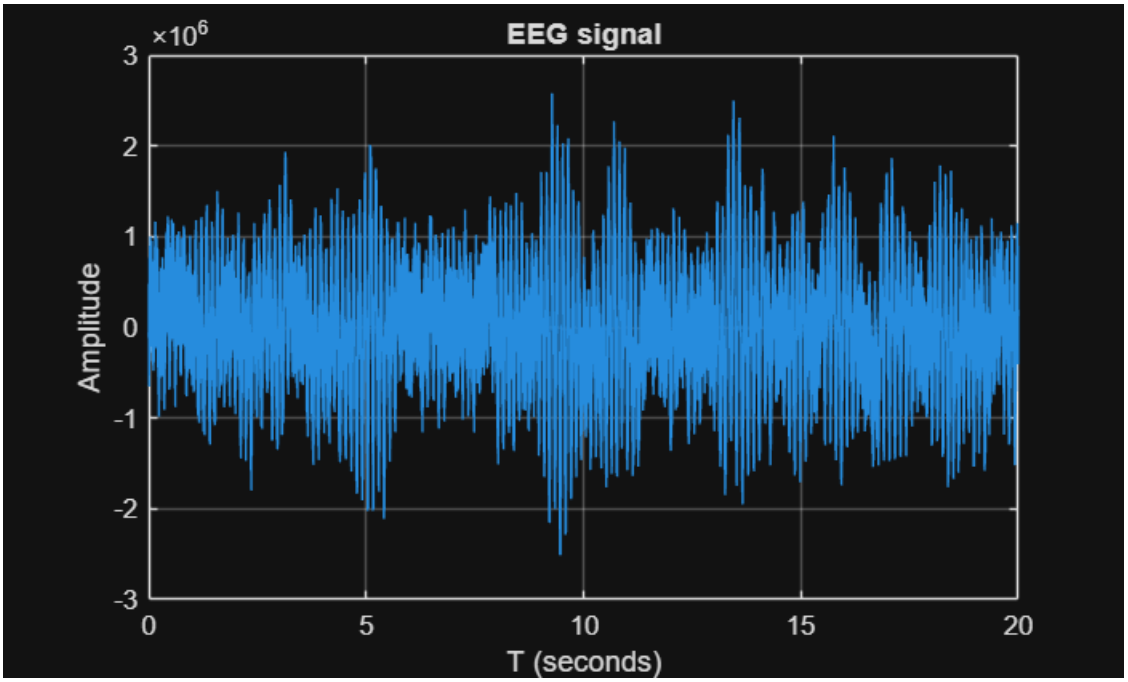


Figure 10. Expected results.

```
figure,plot(t,y);xlim([0 20]);xlabel("T (seconds)");ylabel("Amplitude");title("EEG signal");grid on;
```



QUESTION 6 - Manipulating Biomedical Signals (2 pts)

```
clear
```

Electrocardiogram signal

Electrocardiogram (ECG) recordings capture the heart's electrical activity through non-invasive surface electrodes. In time-domain analysis, the main goal is to examine the morphology and timing of the cardiac waveform, including features such as the P wave, QRS complex, and T wave, which are useful for assessing heart rhythm and identifying abnormalities.

Ten ECG signals are available, stored in the file `ECG_data.mat`, corresponding to an acquisition using a 12-lead configuration. The sampling frequency used is also available.

a) **(0.5 pts)** Load the `ECG_data.mat` file using the `load()` function (see MATLAB documentation).

Obtain the following values:

- Sampling frequency (in **`kHz (kiloHertz)`** units).
- Sampling period of the recording (in **`ms (milliseconds)`** units).
- Time duration of the recording (in **`s (seconds)`** units).

NOTE: The file contains ten channels of data. Consider a single channel (any of them).

```
s= load("ECG_data.mat")
```

```
s = struct with fields:
    raw_ecg: [10×21600 double]
         fs: 360
```

```
fieldnames(s)
```

```
ans = 2×1 cell
'raw_ecg'
'fs'
```

```
y= s.raw_ecg(2,:);
Fs= 360;
Fs_kHz=360/1000
```

```
Fs_kHz =
0.3600
```

```
%Ts=1/Fs
Ts= 1/Fs;
Ts_ms2= Ts*1000
```

```
Ts_ms2 =
2.7778
```

```
%n=0:Nm-1;
Nm= length(y);
n1=0:Nm-1;
%t=n*Ts
t = n1* Ts; % Time vector for the impulse sequence
```

b) **(0.5 pts)** Plot the ECG signal by using the `plot()` function, employ a small window (from 0 to 8 seconds). Customize it by setting `xlabel(), ylabel(), xlim(), ylim(), title()` properly. Can you recognize some pattern in the plotted signal? Find the heart-beat in beats-per-minute (BPM).

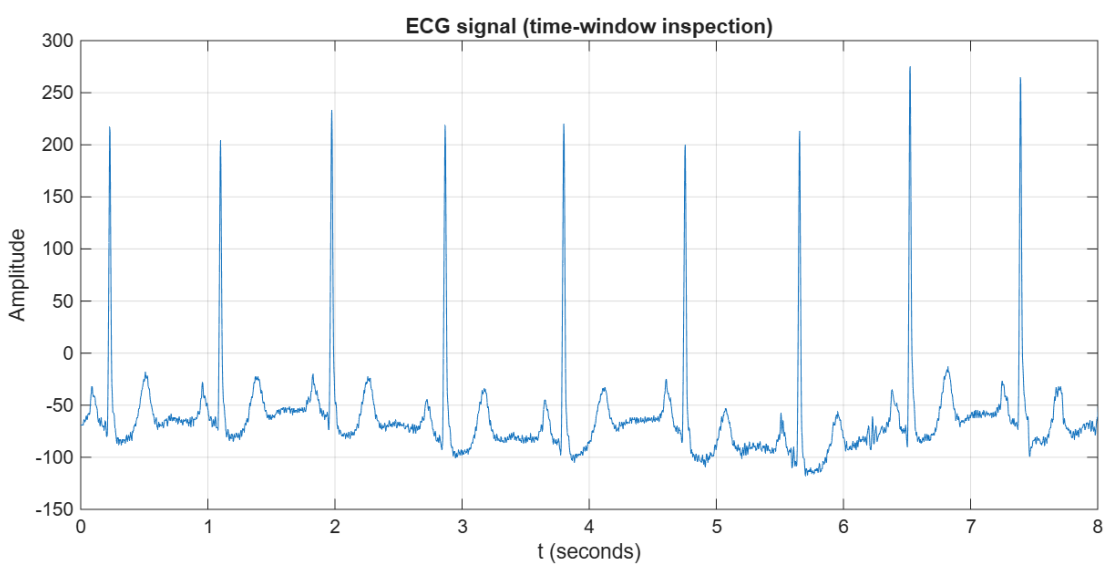
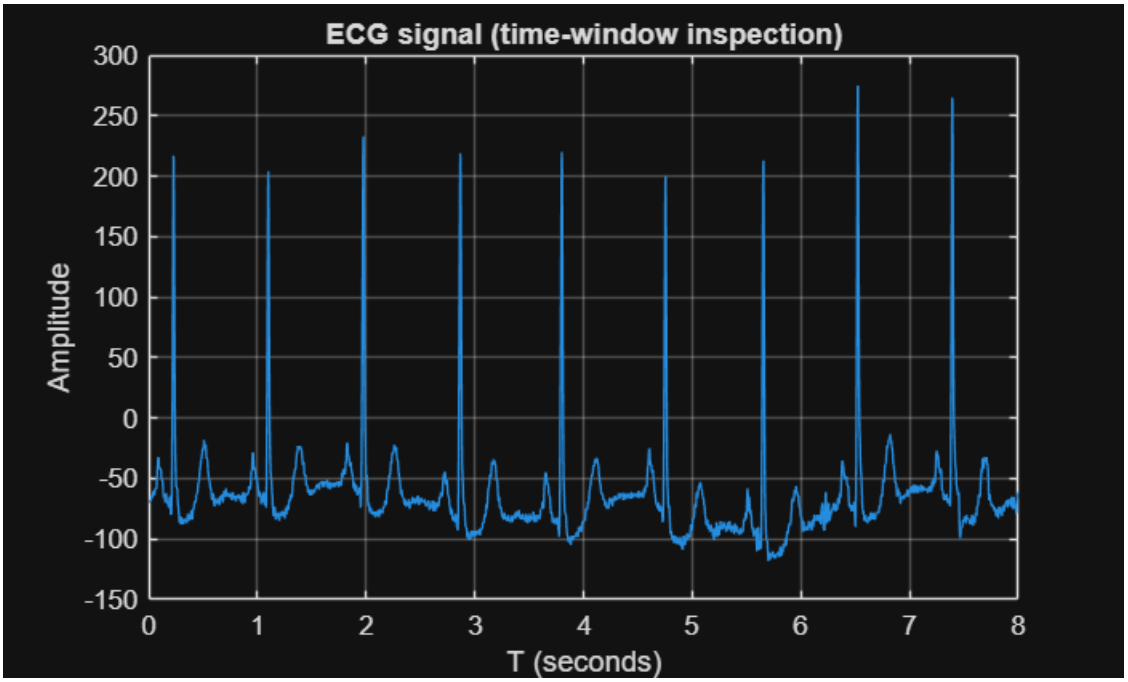


Figure 11. Expected results.

```
figure,plot(t,y);xlim([0 8]);xlabel("T (seconds)");ylabel("Amplitude");title("ECG signal (time-window inspection)");grid on;
```



Electromyogram signal

```
clear
```

Electromyogram (EMG) signals allow for the non-invasive recording of the electrical activity produced by skeletal muscles. Time-domain analysis is essential for identifying muscle activation patterns during contraction, relaxation, or fatigue, as well as for detecting possible neuromuscular disorders.

Four EMG signals are available, stored in the file `EMG_data.mat`, corresponding to an actual recording acquired using surface electrodes. The sampling frequency used is also available.

a) **(0.5 pts)** Load the `EMG_data.mat` file using the `load()` function (see MATLAB documentation).

Obtain the following values:

- Sampling frequency (in **`kHz (kiloHertz)`** units).
- Sampling period of the recording (in **`ms (milliseconds)`** units).
- Time duration of the recording (in **`s (seconds)`** units).

NOTE: The file contains four channels of data. Consider a single channel (any of them).

```
s= load("EMG_data.mat")
```

```
s = struct with fields:
    raw_emg: [4×9964 double]
         fs: 1000
```

```
fieldnames(s)
```

```
ans = 2×1 cell
'raw_emg'
'fs'
```

```
y= s.raw_emg(3,:);
Fs=1000;
Fs_Khz= Fs/1000
```

```
Fs_Khz =
1
```

```
%Ts=1/Fs
Ts=1/Fs;
Ts_ms= Ts*1000
```

```
Ts_ms =
1
```

```
%n=0:Nm-1
Nm=length(y);
n1=0:Nm-1;
t=n1*Ts
```

```
t = 1×9964
         0    0.0010    0.0020    0.0030    0.0040    0.0050    0.0060    0.0070 ···
```

b) **(0.5 pts)** Plot the EMG signal by using the `plot()` function, employ a small window (from 0 to 4 seconds). Customize it by setting `xlabel(), ylabel(), xlim(), ylim(), title()` properly.

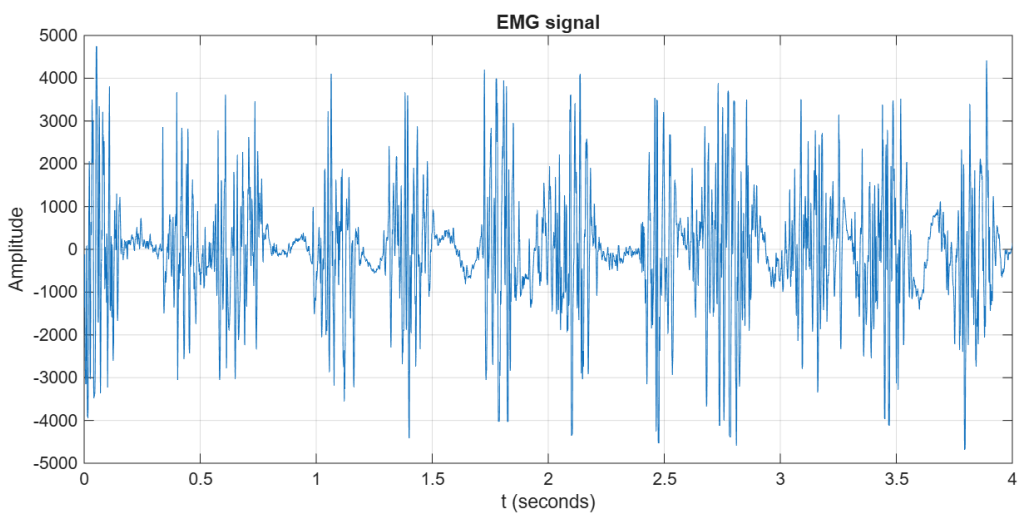
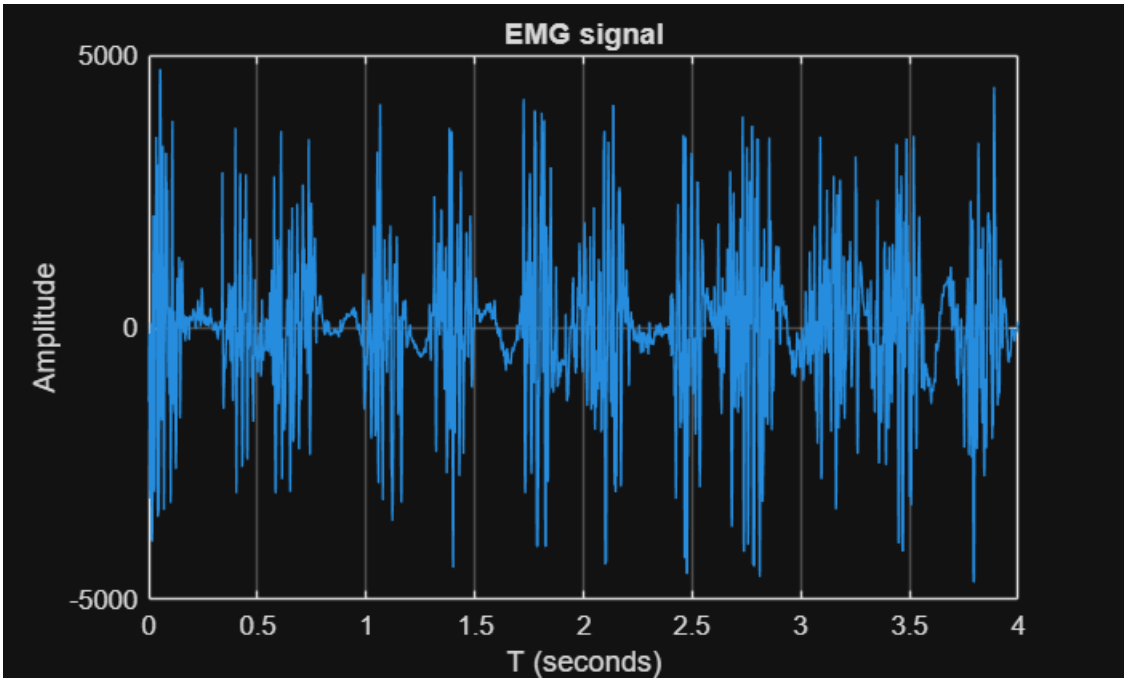


Figure 12. Expected results.

```
figure,plot(t,y);xlim([0 4]);xlabel("T (seconds)");ylabel("Amplitude");title("EMG signal");grid on;
```



QUESTION 7 - Elementary Discrete-Time Signals and properties (6 pts)

```
clear
```

Elementary Signals and Transformations

NOTE: Elementary sequences such as u*nit impulse, unit step, discrete-time sinusoidal and discrete-time complex exponential* are infinite sequences by definition.

- **Unit Impulse**

$$\delta[n] = \begin{cases} 1 & n = 0 \\ 0 & n \neq 0 \end{cases}$$

- **Unit Step**

$$u[n] = \begin{cases} 1 & n \geq 0 \\ 0 & n < 0 \end{cases}$$

- **Discrete-time sinusoidal**

$$x_1[n] = A_1 \cdot \sin(\omega_1 n + \phi_1)$$
$$x_2[n] = A_2 \cdot \cos(\omega_2 n + \phi_2)$$

- **Discrete-time complex exponential**

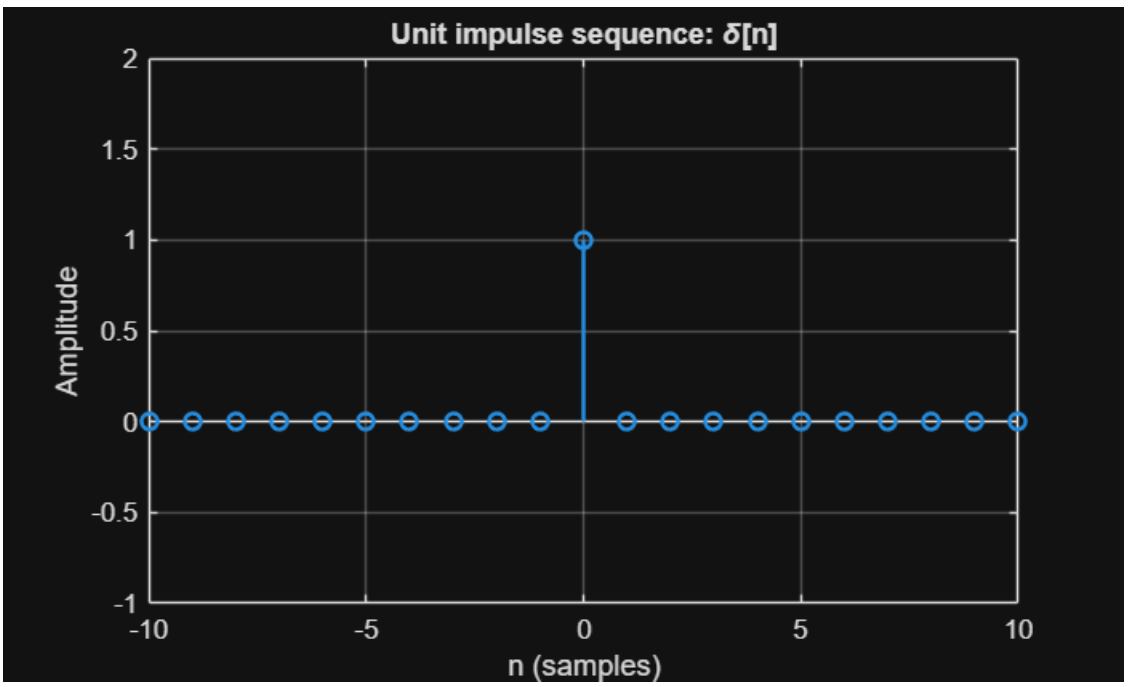
$$x[n] = A \cdot e^{j(\omega n + \phi)} = A \cdot [\cos(\omega n + \phi) + j.\sin(\omega n + \phi)]$$

However, it's only possible to represent finite sequences in MATLAB. For this reason, a particular interval (a set of values for n, where n is the independent variable) must be considered for representing these signals.

Example: Consider the definition interval given by: $n \in [-10, 10]$. Implement a finite u*nit impulse*

```
n = -10:10;
unit_imp = (n == 0);

figure;
stem(n,unit_imp, 'Linewidth',1.5)
title('Unit impulse sequence: \delta[n]')
xlabel('n (samples)');
xlim([-10 10]);
ylabel('Amplitude');
ylim([-1 2]);
grid on;
```



A **scaled and shifted Unit Impulse** can be expressed as:

$$x[n] \;=\; A \cdot \delta\,[n-k] = \begin{cases} A & n = k \\ 0 & n \neq k \end{cases}, \text{ where :}$$

$A:$ scaling factor $(A \in \Re)$
$k:$ time $-$ shifting

A **scaled and shifted Unit Step** can be expressed as:

$$x[n] = A \cdot u[n-k] = \begin{cases} A & n \geq k \\ 0 & n < k \end{cases}, \text{ where :}$$

A : scaling factor ($A \in \Re$)
k : time − shifting

Implement the following sequences and plot the each using `xlabel()`, `ylabel()`, `xlim()`, `ylim()`, `title()` properly. Use the stem() function.

a) **(0.5 pts)**

```
clear
```

- $x_1[n] = 4(u[n] - 2u[n-2] + 0.5u[n-6]), \ n \in [-10, 10]$

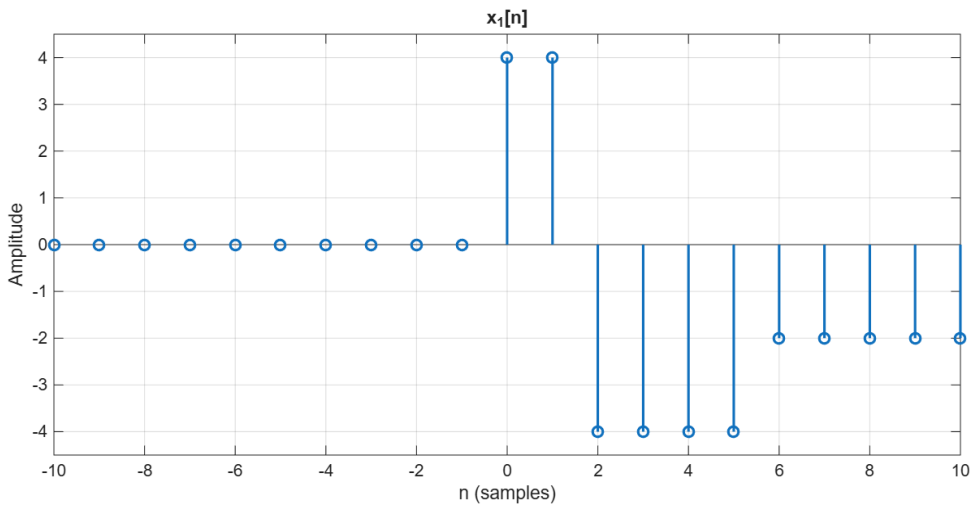
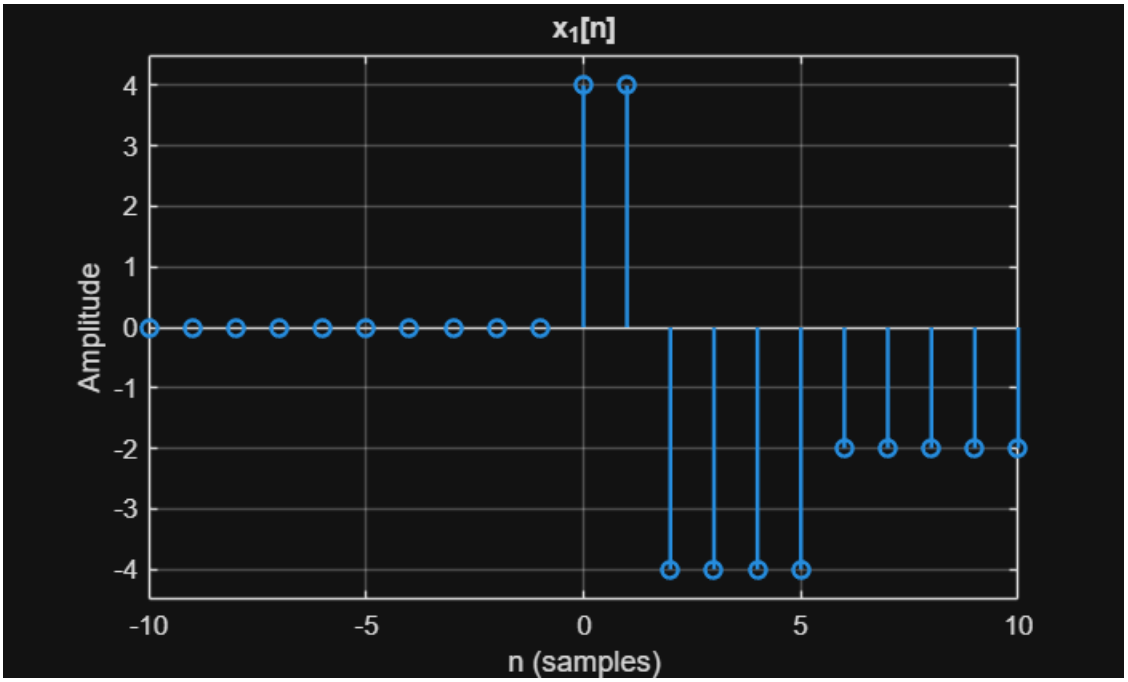


Figure 13. Expected results.

```
n = -10:10;
u_n= (n>=0);
u_n2 = (n>=2);
u_n6= (n>=6);
%building the signal
x1= 4*(u_n-2*u_n2+0.5*u_n6);
%graph
figure,stem(n,x1,'Linewidth',1.5);xlabel("n
(samples)");ylabel("Amplitude");title("x_{1}[n]");xlim([-10 10]);ylim([-4.5
4.5]);grid on;
```



b) **(0.5 pts)**

```
clear
```

- $x_2[n] = 2\delta[n+1] + 4\sin\left(\frac{\pi}{2}n\right) - 3\delta[n+3] + \delta[n], \;\; n \in [-10, 10]$

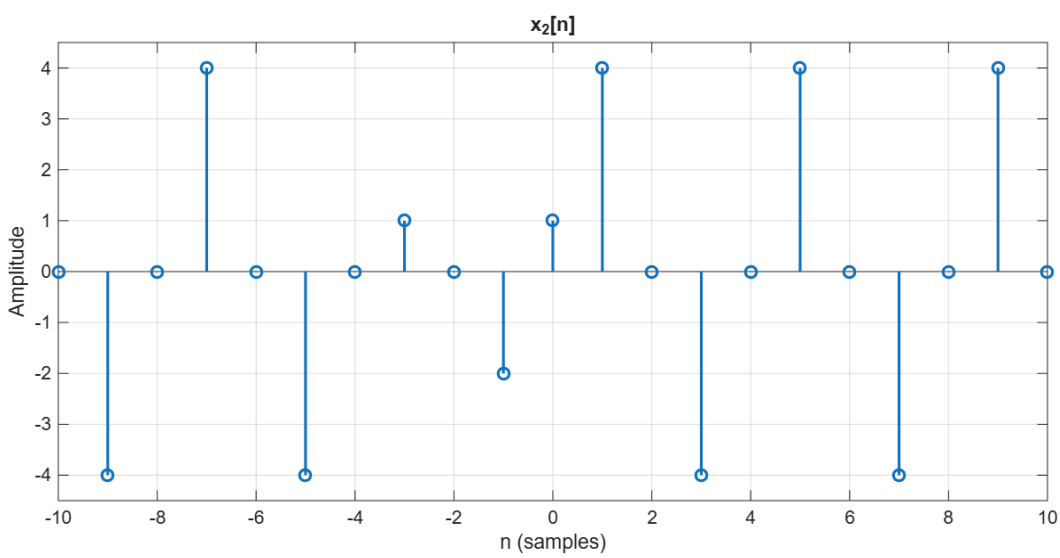
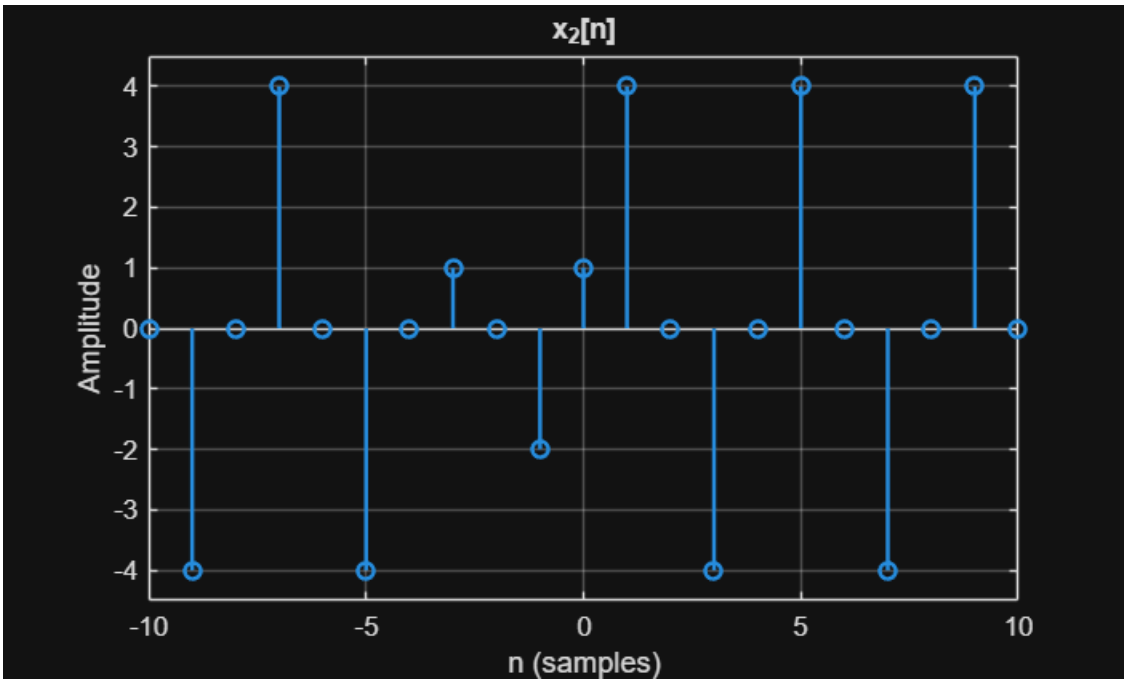


Figure 14. Expected results.

```
n = [-10:10];
%Impulse
delta_n1 = (n==-1);
delta_n3 = (n==-3);
delta_n = (n==0);
%Discrete-time sinusoidal
```

```
sin_n = 4*sin(pi/2 *n);
%Building the signal
x2= 2*delta_n1+sin_n-3*delta_n3+delta_n;
%Graph
figure,stem(n,x2,'Linewidth',1.5);xlabel("n
(samples)");ylabel("Amplitude");title("x_{2}[n]");xlim([-10 10]);ylim([-4.5
4.5]);grid on;
```



c) **(1.5 pts)**

```
clear
```

- $x_M[n] = \sum_{k=0}^{M} a_k \phi_k[n] + \sum_{k=0}^{M} b_k \psi_k[n], \;\; n \in [-10, 100]$

Where:

- $\phi_k[n] = (u[n-kL] - u[n-kL-W])s_k[n]$
- $\psi_k[n] = (\delta[n-kL] - \delta[n-kL-W])$
- $s_k[n] = \sum_{m=0}^{P} (-1)^m \frac{(\omega_k \tau)^{2m+1}}{(2m+1)!}, \quad \omega_k = 0.12\pi + 0.03\pi k, \quad \tau = n - kL$
- $a_k = \prod_{i=1}^{k} \frac{\alpha}{i}, a_0 = 1, \quad b_k = (-1)^k \frac{\alpha^k}{k+1}$

Consider: $M = 10, \;\; L = 18, \;\; W = 10, \;\; P = 10, \;\; \alpha = 0.85$

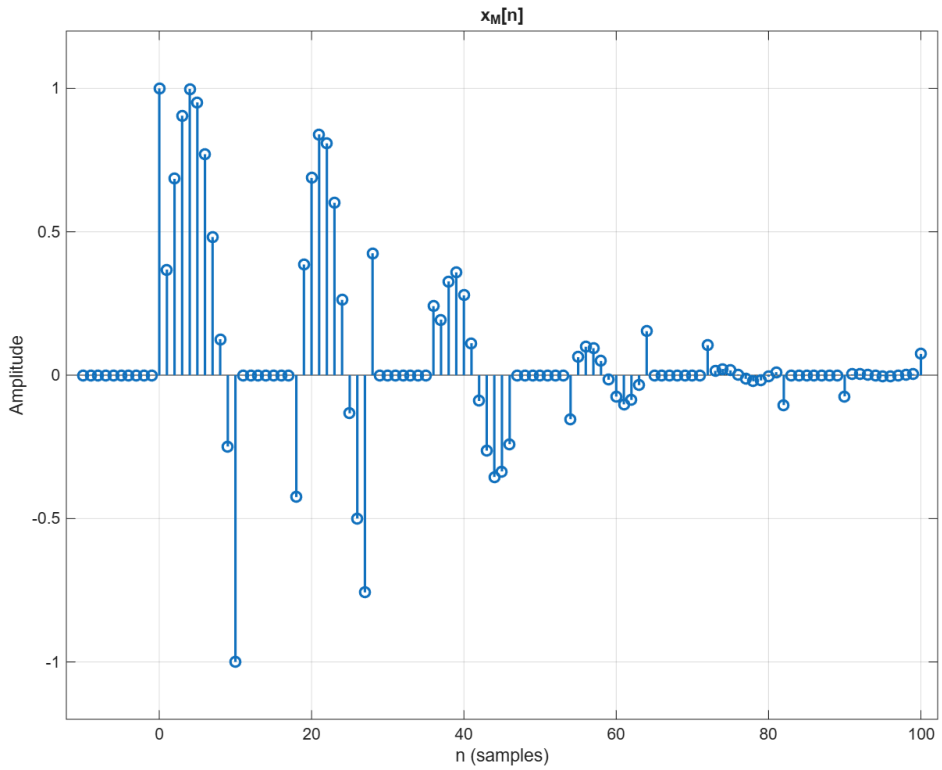
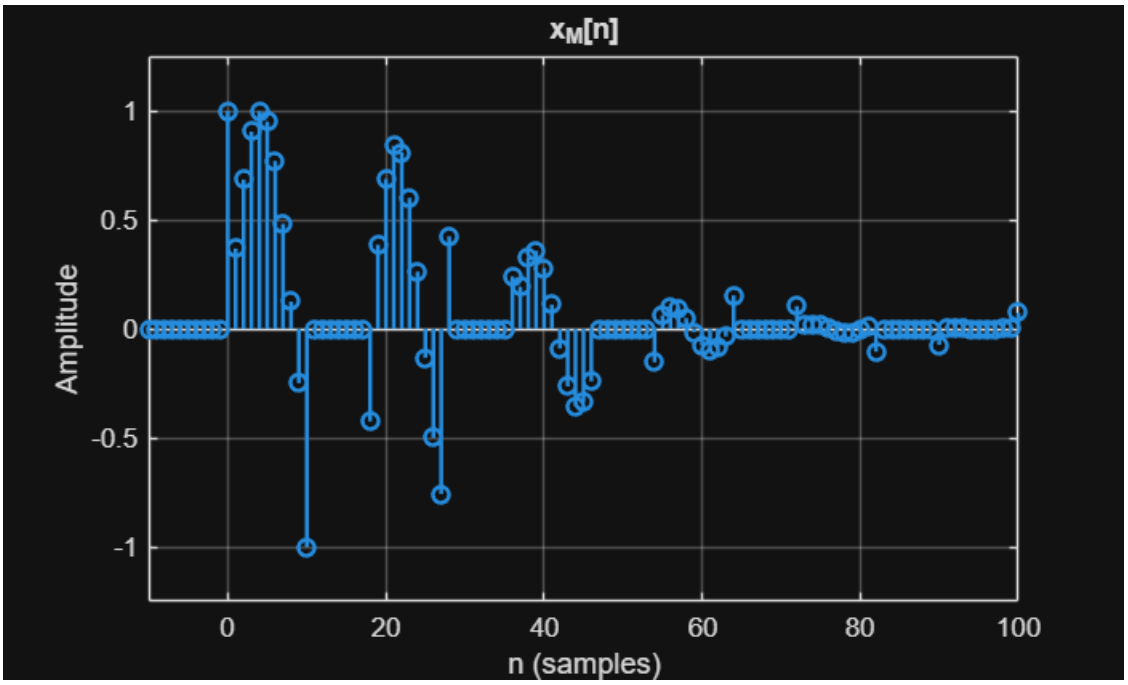


Figure 15. Expected results.

```
n= [-10:100];
%we initialize the signal
xM = zeros(size(n));
%conder:
M= 10;
L=18;
W=10;
P=10;
alpha=0.85;
%loop k
for k = 0:M
    %find ak
    if k==0
        ak=1;
    else
        ak= prod(alpha ./ (1:k));
    end
    %find bk
    bk = ((-1)^k * alpha^k) / (k+1);
    %then tau and wk
    tau = n - k*L;
    wk = 0.12*pi + 0.03*pi*k;
    %find sk[n]
    sk = zeros(size(n));
    for m = 0:P
        sk = sk + ((-1)^m * (wk*tau).^(2*m+1)) / factorial(2*m+1);
```

```
    end
    %Now find phi[n]
    phi = ((n >= k*L) & (n < k*L + W)) .* sk;
    %Also find psi[n]
    psi = (n == k*L) - (n == (k*L + W));
    %Finally accumulate
    xM = xM + ak*phi + bk*psi;
end
%Graph
figure,stem(n,xM,'Linewidth',1.5);xlabel("n
(samples)");ylabel("Amplitude");title("x_{M}[n]");xlim([-10 100]);ylim([-1.25
1.25]);grid on;
```



d) **(1.5 pts)**

```
clear
```

- $x[n] = \sum_{k=0}^{M} a_k(p_k[n] - 1)(u[n - n_k] - u[n - n_k - L]), \;\; n \in [-40, 80]$

Where:

- $p_k[n] = p_{k-1}[n](1 + \beta_k \cdot s_k[n]), \; k \geq 1 \qquad \wedge \qquad p_0[n] = 1 + \beta_0 \cdot s_0[n]$
- $\beta_k = \dfrac{0.7}{k+1}$
- $n_k = 18k - 60$
- $s_k[n] = e^{-\lambda_k \tau} \cos(\omega_k \tau + \eta_k \tau^2), \quad \lambda_k = 0.02 + 0.01k, \quad \eta_k = 0.0008(k+1), \quad \omega_k = 0.14\pi + 0.025\pi k, \quad \tau = n - kL$

- $a_k = a_{k-1}\dfrac{k+2}{2k+1}\alpha, \quad a_0 = 1$

Consider: $M = 4, \;\; L = 16, \;\; \alpha = 0.92$

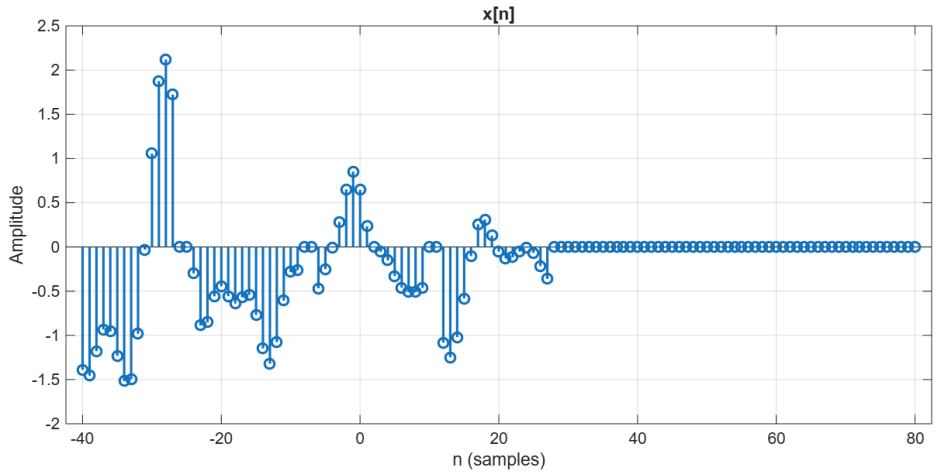


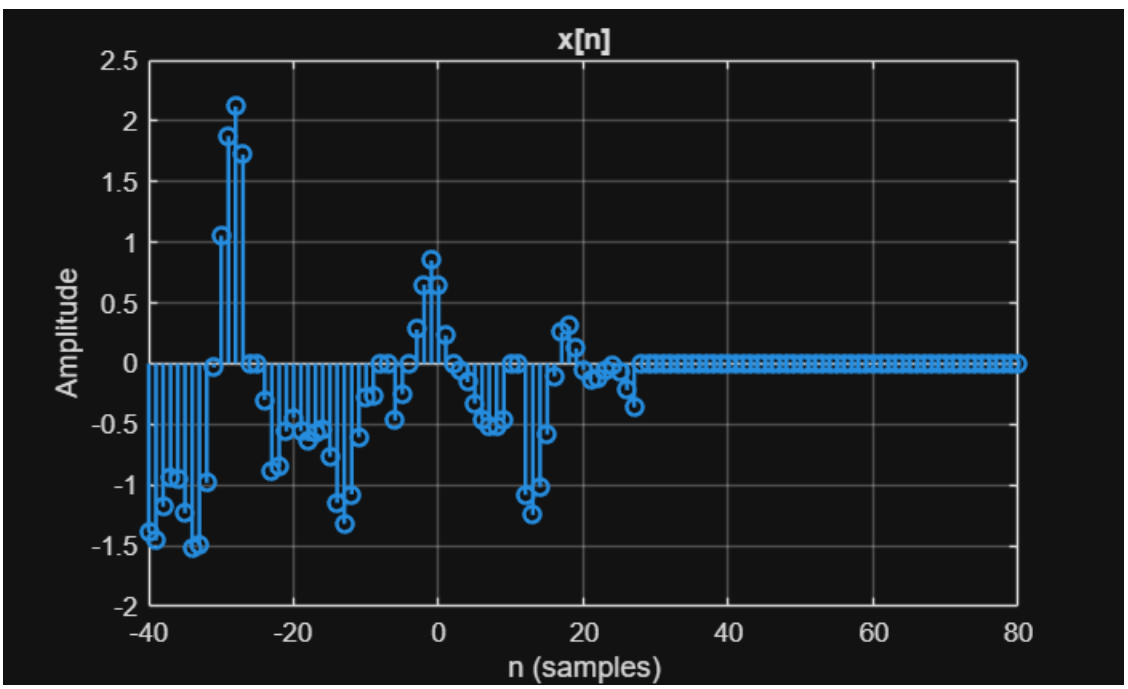
Figure 16. Expected results.

```
n = [-40:80];
x= zeros(size(n));
%Consider
M= 4;
L= 16;
alpha=0.92;
%we initialize
p_prev = zeros(size(n));
a_prev = 1;
%We star the loop
for k= 0:M
    %we calculate beta_k
    beta_k= 0.7/(k+1);
    %Now we calculate nk
    nk= 18*k-60;
    %Then we calculate tau
    tau= n-k*L;
    %k dependent parameters
    lambda_k = 0.02 + 0.01*k;
    eta_k = 0.0008*(k+1);
    wk = 0.14*pi + 0.025*pi*k;

    %Then we calculate s_k[n]
    sk = exp(-lambda_k * tau) .* cos(wk*tau + eta_k*tau.^2);
    %Then we calculate s_k[n]
    if k==0
        pk= 1+ beta_k * sk;
    else
        %.* multiply the arrays and multiplying corresponding elements
```

```
        pk = p_prev .*(1 + beta_k*sk);
    end
    %Then we calculate a_k
    if k==0
        ak=1;
    else
        ak = a_prev * ((k+2)/(2*k+1)) * alpha;
    end
    %Now calculate the other part (n >= nk) & (n < nk + L);
    part2= (n >= nk) & (n < nk + L);

    %Finally accumulate
    x = x + ak * (pk - 1) .* part2;
    %Don't forget to save for the next ite
    p_prev = pk;
    a_prev = ak;
end
%Graph
figure,stem(n,x,'Linewidth',1.5);xlabel("n
(samples)");ylabel("Amplitude");title("x[n]");xlim([-40 80]);ylim([-2 2.5]);grid on;
```



e) **(1 pt)** Recreate the following signal shown in the figure using only scaled and/or shifted step functions.

```
clear
```

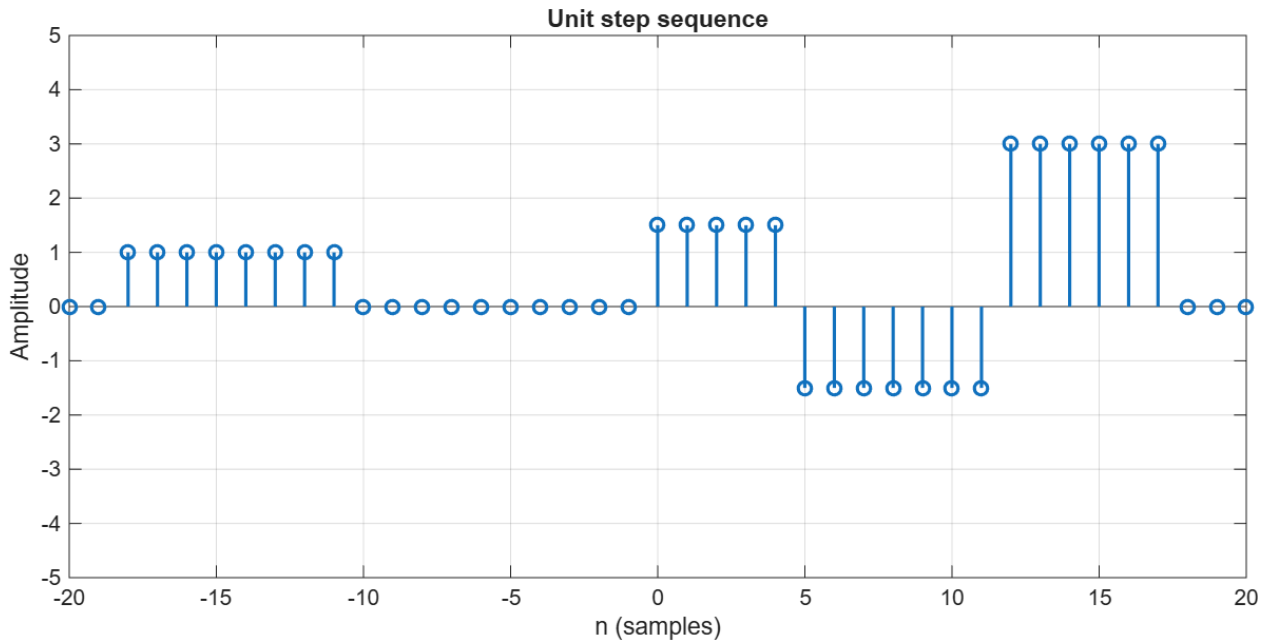
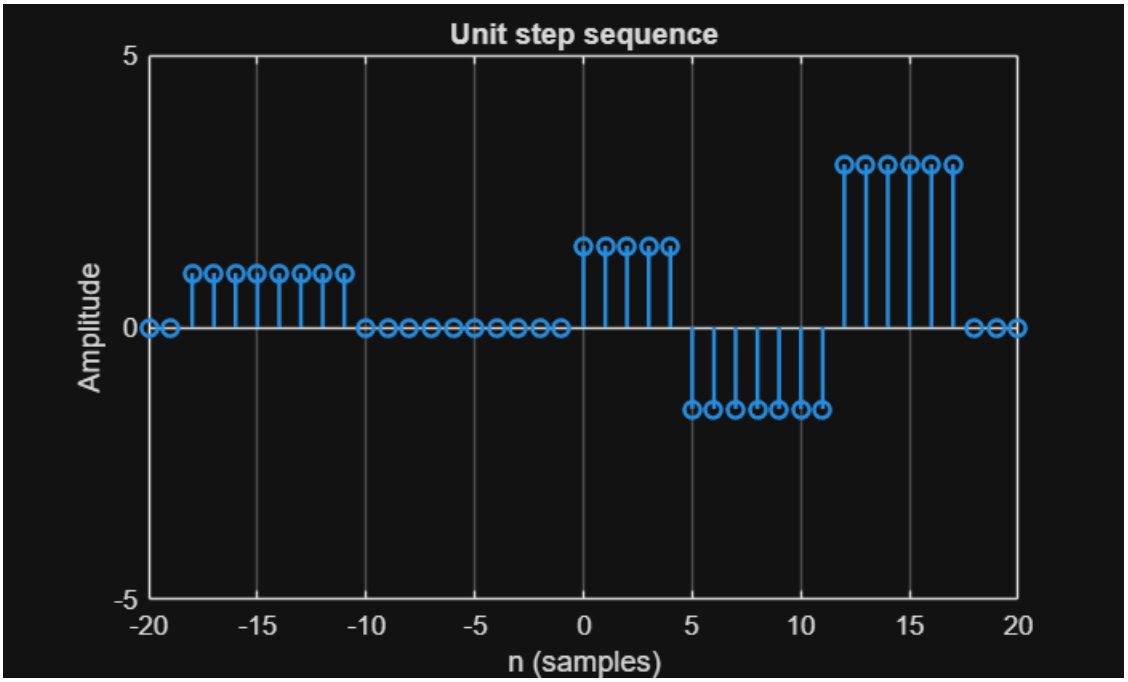


Figure 17. Expected results.

```
n= -20:20;
%we initialize the signal
x = zeros(size(n));

%use only shifted/scaled step functions
%use only shifted/scaled step functions
x = 1*((n>=-18) - (n>=-10)) + 1.5*((n>=0) - (n>=5)) - 1.5*((n>=5) - (n>=12)) +
3*((n>=12) - (n>=18));

%Graph
figure;stem(n,x,'LineWidth',1.5);xlabel("n
(samples)");ylabel("Amplitude");title("Unit step sequence");xlim([-20 20]);ylim([-5
5]);grid on;
```



f) **(1 pt)** Recreate the following signal shown in the figure using only scaled and/or shifted step functions. You must also use a "for" loop.

```
clear
```

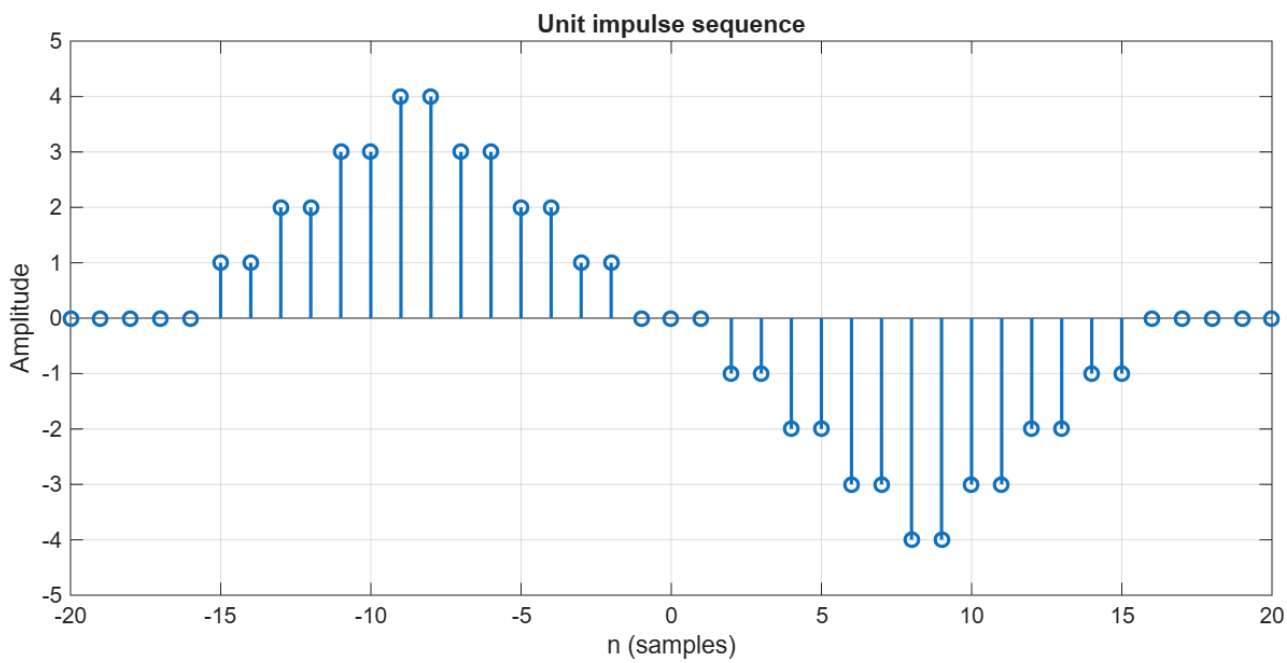
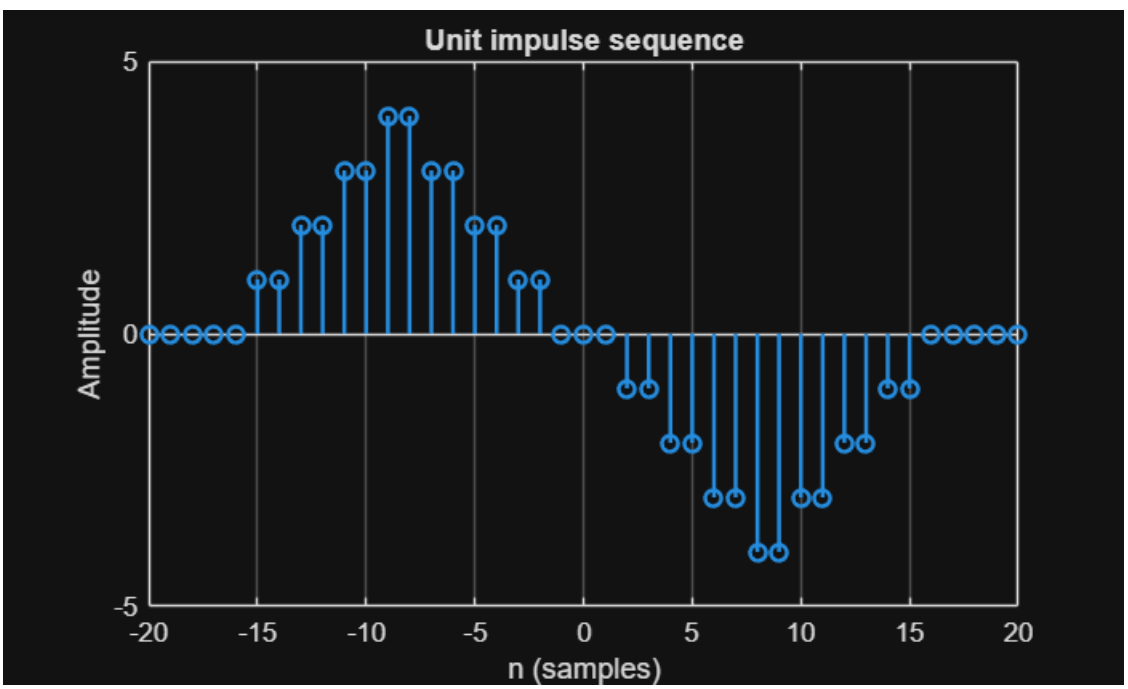


Figure 18. Expected results.

```
%Define the time index range
```

```
n = -20:20;
x = zeros(size(n));

%Define the Step Events
step_indices = [-15, -13, -11, -9, -7, -5, -3, -1, 2, 4, 6, 8, 10, 12, 14, 16];
jump_values  = [ 1,   1,   1,  1, -1, -1, -1, -1, -1, -1, -1, -1,  1,  1,  1,  1];

%Loop
%Iteratively add shifted unit steps to build the final sequence
for i = 1:length(step_indices)
    current_n = step_indices(i);
    amplitude = jump_values(i);

    unit_step = (n >= current_n);
    x = x + amplitude * unit_step;
end

%Graph
figure;
stem(n, x, 'LineWidth', 1.5);grid on;xlim([-20 20]); ylim([-5 5]);xlabel("n
(samples)");ylabel("Amplitude");title("Unit impulse sequence")
```